

AIC-2101 Lab Guide

Spec-Driven Development with OpenSpec



Part of
Accenture

© 2026 Ascendient, LLC

Revision 1.1.0 published on 2026-05-28

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC
1 California Street Suite 2900
San Francisco, CA 94111
<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com
Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

● You may:

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

● You may not:

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

🕒 Important

The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

Table of Contents

1. Foundations & First Cycle	6
1.1. Prerequisites.....	7
1.2. Introduction	8
1.3. Exercise 1: Set Up the BeanBotics Starter App	8
1.4. Exercise 2: Pick Your Agent.....	10
1.5. Exercise 3: Initialize OpenSpec.....	12
1.6. Exercise 4: Explore the Codebase with Your Agent	15
1.7. Exercise 5: Your First Propose-Apply-Archive Cycle	16
1.8. Reflection Questions	20
1.9. Common Issues.....	20
1.10. Challenge Exercise (Optional)	21
1.10.1. Propose a Second Feature.....	21
1.11. Context Window Management Tips.....	22
1.11.1. How Do I Know When to Clear My Context Window?.....	22
1.11.2. How Do I Transition Without Losing Important Context?	23
2. Exploring, Proposing & Specifying.....	24
2.1. Prerequisites.....	25
2.2. Introduction	25
2.3. Exercise 1: Open This Lab's BeanBotics Folder.....	26
2.4. Exercise 2: Explore the Order System.....	27
2.5. Exercise 3: Propose the Order Status Lifecycle	30
2.6. Exercise 4: Review and Refine the Artifacts	31
2.7. Exercise 5: Apply, Verify, and Archive	34
2.8. Reflection Questions	37
2.9. Common Issues.....	38
2.10. Challenge Exercise (Optional)	39
2.10.1. Write a Scenario From Scratch.....	40
2.10.2. Compare Spec Quality.....	40
2.11. Context Hygiene Recap	40
3. Implementation Patterns & Debugging.....	42
3.1. Prerequisites.....	43
3.2. Introduction	43
3.3. Exercise 1: Set Up Your Working Copy.....	44
3.4. Exercise 2: Drink Customizations. The Fast Cycle	46

3.5. Exercise 3: Order Confirmation & Receipt. Dependency Handling	49
3.6. Exercise 4: Debugging Spec Ambiguity	52
3.7. Exercise 5: Review the Accumulated System	54
3.8. Reflection Questions	55
3.9. Common Issues.....	55
3.10. Challenge Exercise (Optional)	57
3.10.1. Manually Add a Tax Scenario	57
3.10.2. Compare Your Two Cycles	57
3.11. Context Hygiene Recap	58
3.12. Extra Material: Design Doc Abstraction Level.....	59
4. Expanded Workflow & Spec Lifecycle	61
4.1. Prerequisites.....	62
4.2. Introduction	62
4.3. Exercise 1: Set Up Your Working Copy.....	63
4.4. Exercise 2: Switch to the Expanded Profile	65
4.5. Exercise 3: Scaffold and Build the Proposal.....	69
4.6. Exercise 4: Build Specs and Design with Review Gates	72
4.7. Exercise 5: Implement and Verify	75
4.8. Exercise 6: Archive and Review the Spec Suite	78
4.9. Reflection Questions	79
4.10. Common Issues.....	79
4.11. Challenge Exercise (Optional)	82
4.11.1. Calculate Margins for Multiple Orders.....	82
4.11.2. Write a Spec Scenario by Hand.....	82
4.12. Context Hygiene Recap	83
5. Real-World Patterns & Spec Management.....	84
5.1. Prerequisites.....	85
5.2. Introduction	85
5.3. Exercise 1: Set Up Your Working Copy.....	86
5.4. Exercise 2: Fix Without SDD.....	88
5.5. Exercise 3: Identify Pricing Assumptions in the Spec Suite.....	89
5.6. Exercise 4: Write Specs by Hand.....	90
5.7. Exercise 5: Implement Dynamic Pricing.....	101
5.8. Exercise 6: Trace the Full Evolution	103
5.9. Reflection Questions	104
5.10. Common Issues.....	105
5.11. Challenge Exercise (Optional)	106
5.11.1. Add a Third Pricing Tier.....	106
5.11.2. Calculate the Margin Impact	107
5.12. Context Hygiene Recap	107

Foundations & First Cycle

LAB 1

-
- ✓ Set up and run the BeanBotics starter application
 - ✓ Initialize OpenSpec in an existing project and configure project context
 - ✓ Use an AI coding agent to explore a codebase and identify missing functionality
 - ✓ Complete a full propose-apply-archive cycle using OpenSpec's core workflow
 - ✓ Practice context hygiene by clearing context between exploration and specification work

1.1. Prerequisites

Before starting this lab, ensure you have the following installed:

- **Python 3.10+** with `pip`
- **Node.js 18+** with `npm` (for OpenSpec CLI)
- **VS Code** (or your preferred editor) with one of the seven supported AI coding agents installed and signed in (see **Command syntax** below for the list)
- A terminal (the VS Code integrated terminal works well)

Tip

If you have not completed the setup guide, do so before proceeding. The setup guide covers installing all prerequisites.

Note

Command syntax depends on your agent
Throughout this course, when prose says "invoke the **propose** command," substitute the command name into the pattern for your agent:

Agent	Example: propose
Claude Code	<code>/opsx:propose</code>
Cursor, Windsurf, GitHub Copilot, Google Antigravity	<code>/opsx-propose</code>
Amazon Q, AWS Kiro	<code>@opsx-propose</code>

Apply the same pattern to every other command (`apply` , `archive` , `continue` , ...).

Note

If you use an agent outside these seven, check `openspec/AGENTS.md` for the exact syntax. Some agents use `openspec-` instead of `opsx-` as the command prefix.

1.2. Introduction

In this lab, you will experience the full Spec-Driven Development workflow for the first time. You will start by running a small web application called **BeanBotics**: an AI-themed coffee ordering system. After exploring the application and understanding what it does, you will use OpenSpec to propose, implement, and archive your first spec-driven feature.

The feature you will build is **drink size selection**. Right now, every order is placed as a medium. The menu data already includes pricing for small, medium, and large sizes, but the user interface does not let customers choose. By the end of this lab, customers will be able to select their preferred size before ordering.

This is deliberately a small change. The goal is not the feature itself. It is learning the workflow. You will use this same propose-apply-archive cycle throughout the course for progressively larger changes.

1.3. Exercise 1: Set Up the BeanBotics Starter App

In this exercise, you will open the lab's BeanBotics folder in VS Code, create a virtual environment, install dependencies, and verify the app runs.

1. **Open the lab's BeanBotics folder in VS Code.** The course materials include a starter application at `~/LabFiles/foundations-first-cycle/beanbotics/`. Open that folder as its own VS Code workspace:

```
code ~/LabFiles/foundations-first-cycle/beanbotics
```

⚠ Important

For the rest of this lab, **all work happens in this VS Code window** with `beanbotics` as the project root. Your AI agent and the OpenSpec commands only work when the editor is opened to the directory where OpenSpec is initialized. If you have other editor windows open for other

projects, that is fine. Just make sure your lab work happens in this window.

2. **Set up the Python environment.** Open a terminal inside the VS Code window (`Ctrl` + ```) and run:

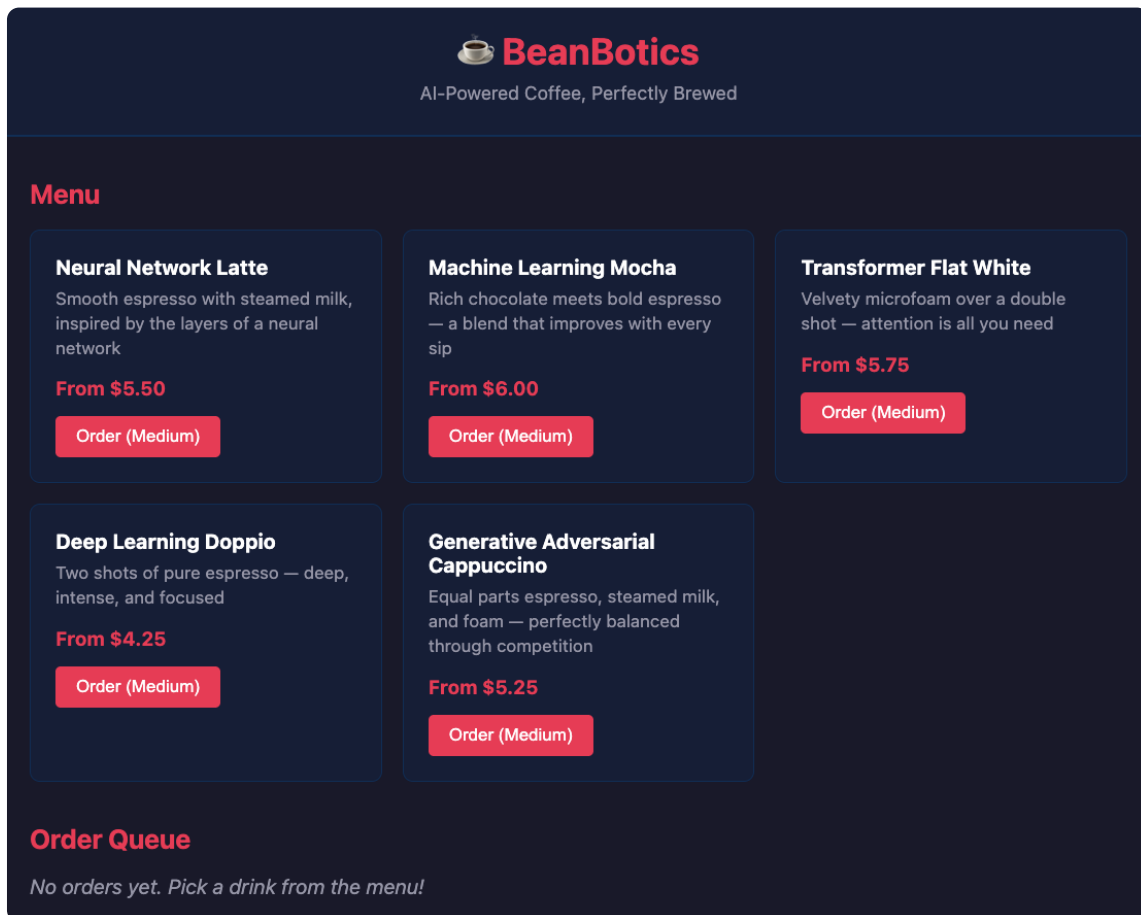
```
uv venv
uv pip install -r requirements.txt
```

3. **Run the application.**

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

This starts a FastAPI server on <http://localhost:8000>.

4. **Verify the UI.** Open <http://localhost:8000> in your browser. Your UI should look like this:



Notice the **Order (Medium)** buttons. Every drink can only be ordered in one size. You will change this later in the lab.

5. **Explore the UI.** Open `http://localhost:8000` in your browser. You should see:

- A **menu section** with five AI-themed coffee drinks displayed as cards
- Each card shows the drink name, description, price, and an **Order (Medium)** button

- An **order queue section** that starts empty

Try placing a few orders and cancelling one. Notice that:

- Every order is placed as **medium**: there is no way to choose a size
- The price shown is the medium price only
- The order queue updates after each action

Leave the server running in the terminal. You will use it throughout the lab; if it stops at any point, restart it with

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

Note

If port 8000 is already in use, change the `--port` argument in the `uv run uvicorn ...` command above, or stop whatever is using that port.

1.4. Exercise 2: Pick Your Agent

You will work with one AI coding agent throughout this course. Pick one now and stay with it. Switching mid-course leaves half-configured projects behind. The seven supported agents fall into three invocation patterns; pick the tab below that matches the agent you intend to use.

Slash-colon (Claude Code)

1. **Install:** the **Claude Code** extension in VS Code (or use the Claude Code CLI)
2. **Sign in:** `Ctrl + Shift + P` → **Claude Code: Sign In**
3. **Verify the chat panel opens:** `Ctrl + Esc` in VS Code
4. **Commands look like** `/opsx:propose`

Slash-hyphen (Cursor, Windsurf, GitHub Copilot, Google Antigravity)**1. Install your agent:**

- **Cursor** and **Windsurf**: download the standalone editor from cursor.com / windsurf.com
- **GitHub Copilot**: install the **GitHub Copilot Chat** extension in VS Code
- **Google Antigravity**: download the Antigravity IDE from antigravity.google

2. Sign in: Cursor and Windsurf use their own accounts; Copilot uses your GitHub account; Antigravity uses your Google account**3. Verify the chat panel opens:**

- **Cursor** and **Windsurf**: `Ctrl + L`
- **GitHub Copilot**: `Ctrl + Alt + I`
- **Antigravity**: the chat panel is in the IDE sidebar

4. Commands look like `/opsx-propose`***@-mention (Amazon Q, AWS Kiro)*****1. Install your agent:**

- **Amazon Q Developer**: install the **Amazon Q** extension in VS Code
- **AWS Kiro**: download the Kiro IDE from kiro.dev

2. Sign in: both use your AWS Builder ID (or IAM Identity Center for enterprise users)**3. Verify the chat panel opens:**

- **Amazon Q**: click the **Q** icon in the VS Code activity bar
- **AWS Kiro**: the chat panel is open by default in the Kiro IDE

4. Commands look like `@opsx-propose` (not slash-prefixed)**Note**

From here on, when this lab says "invoke the **propose** command," use the pattern for your agent from the **Command syntax** block at the top of this lab.

1.5. Exercise 3: Initialize OpenSpec

In this exercise, you will verify the OpenSpec CLI is available, initialize it in the BeanBotics project, and examine what it creates.

1. **Verify OpenSpec is installed.** OpenSpec was installed on the lab VM during course setup. In a **new terminal** (keep the server running in the first one), confirm it is available:

```
openspec --version
```

You should see `1.2.0`. If the command is not found or the version differs, the lab VM setup did not complete successfully. Alert your instructor before proceeding.

2. **Initialize OpenSpec in the project.** In the VS Code terminal (which is already open inside `beanbotics/`), run:

```
openspec init
```

When prompted, select the AI tool you chose in Exercise 2. The **core** profile is used by default. You do not need to select it.

ⓘ Important

After `openspec init` completes, reload the VS Code window so your AI agent picks up the newly-installed OpenSpec commands. Open the command palette with `Ctrl + Shift + P`, type **Developer: Reload Window**, and select it.

The reload stops the BeanBotics server because it runs inside the integrated terminal. After the reload, open a new terminal with `Ctrl + `` and restart the server:

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

The OpenSpec commands only work when your editor is open to the directory where you ran `openspec init`.

3. **Examine the generated structure.** After initialization, explore what OpenSpec created. In a **terminal** (not your agent's chat panel), run:

```
ls -la openspec/
```

You should see:

- `openspec/`. The specification directory with `specs/`, `changes/`, and optional config files
- `openspec/specs/`. Currently empty. This is where the source-of-truth specifications will live after your first archive
- `openspec/changes/`. Currently empty. This is where active work will be tracked

OpenSpec also wrote command files into your agent's own config directory. List that directory to see them. Which directory depends on the agent you selected:

Table 1. How agents store their context

Agent	Command directory	Invocation
Claude Code	<code>.claude/commands/opsx/</code>	<code>/opsx:propose</code>
Cursor	<code>.cursor/commands/</code>	<code>/opsx-propose</code>
Windsurf	<code>.windsurf/workflows/</code>	<code>/opsx-propose</code>
GitHub Copilot	<code>.github/prompts/</code>	<code>/opsx-propose</code>
Google Antigravity	<code>.agent/workflows/</code>	<code>/opsx-propose</code>
Amazon Q	<code>.amazonq/prompts/</code>	<code>@opsx-propose</code>
AWS Kiro	<code>.kiro/prompts/</code>	<code>@opsx-propose</code>

Your agent reads its own directory; the others are ignored. The course's checkpoint folders ship all seven trees pre-generated, so you can switch agents between labs without re-running `openspec init`.

Notice there are now **two** sources giving context to your coding agent:

- **AGENTS.md**: already in the project. You (the developer) write and maintain this. It tells your agent about the project: tech stack,

structure, conventions. `AGENTS.md` is an emerging cross-agent convention that most listed agents read directly; a few (like Claude Code) also read a tool-specific file such as `CLAUDE.md`, but `AGENTS.md` is the agent-neutral source of truth in this course.

- **Agent-specific commands:** generated by OpenSpec in the directory above (e.g., `.claude/commands/opsx/` for Claude Code, `.cursor/commands/` for Cursor). These tell your agent how to work with the OpenSpec framework: available commands, where specs live, workflow conventions.

The key distinction: `AGENTS.md` is yours to edit. The agent-specific command files are managed by OpenSpec. Do not edit them by hand. If you need to update them, run `openspec update`.

4. **Configure the project context.** `openspec init` created `openspec/config.yaml` for you. Open it in VS Code and replace its contents with:

```
schema: spec-driven

context: |
  BeanBotics is an AI-themed coffee ordering web application.
  Tech stack: Python 3.10+, FastAPI, vanilla HTML/CSS/JavaScript
  frontend.
  Backend: FastAPI app in backend/ with service layer pattern.
  Frontend: Single-page app in frontend/ using fetch API for all
  server calls.
  Data: Menu items loaded from backend/data/menu.json. Orders
  stored in memory.
  No database, no authentication, no payment processing.
  API endpoints: GET /api/menu, POST /api/orders, GET /api/
  orders, DELETE /api/orders/{id}.
```

This context is injected into every artifact that OpenSpec generates, giving the AI agent accurate information about the project without needing to re-discover it each time.

Tip

Keep `config.yaml` context focused on facts the AI cannot easily infer from the code. Tech stack, architectural patterns, and non-obvious constraints are good. Generic best practices are not. The AI already knows those.

1.6. Exercise 4: Explore the Codebase with Your Agent

In this exercise, you will use your AI coding agent conversationally to understand the BeanBotics codebase before proposing any changes. This mirrors the realistic developer workflow: investigate first, then plan.

1. **Open your agent's chat panel.** Open the chat panel for the agent you set up in Exercise 2. The exact way to do this depends on the agent. Consult your agent's documentation if you are unsure. Most agents offer either a sidebar icon or a keyboard shortcut.
2. **Ask your agent to explore.** Ask your agent to examine the project. Here are some good exploratory prompts:

What does this project do? Walk me through the main files and architecture.

Look at the menu data in `backend/data/menu.json`. What drink sizes are available, and how does the frontend currently handle size selection?

What would need to change to let customers choose between small, medium, and large sizes when ordering?

3. **Understand the gap.** Through your exploration, you should discover:
 - The **backend already supports sizes**. The `OrderRequest` model accepts a `size` field, and the `menu.json` file has pricing for small, medium, and large
 - The **frontend hardcodes medium**. The `placeOrder()` function in `script.js` always sends `size: "medium"`, and the order button says "Order (Medium)"
 - The **price display only shows the medium price**. The menu cards show "From \$X.XX" using only the medium price
This means the feature is primarily a **frontend change**: the backend is already ready.
4. **Clear your context.** This is your first **context hygiene** practice. You have been exploring conversationally. Asking questions, reading code, building understanding. That exploration context is useful for *you*, but it will clutter the AI's context window when you move to specification work.

Clear the context now using your agent's clear-context mechanism. Most agents offer either a slash command (such as `/clear` or `/new`), a "new chat" or "new session" button in the chat panel, or both. Consult your agent's documentation if you cannot find it.

This gives you a fresh context window for the next exercise. The specifications you create will be cleaner because the AI agent is not carrying forward noise from the exploration conversation.

Note

Your understanding persists in *your* head. The specifications persist in *files*. Clearing the AI's context does not lose anything important. It just removes conversational noise.

1.7. Exercise 5: Your First Propose-Apply-Archive Cycle

In this exercise, you will use the core OpenSpec workflow to propose, implement, and archive your first spec-driven feature: drink size selection.

1. **Propose the feature.** In your agent's chat panel (with the fresh context from the previous step), invoke the **propose** command (see **Command syntax** above) with the change name `add-drink-size-selection` and this intent text:

```
Add a size selector (small, medium, large) to the order form so
customers can choose a size before placing an order. The
selected
size should drive the displayed price and be sent with the
order.
```

```
Out of scope: drink customizations (milk, flavor, extra shots),
menu changes, new backend endpoints, and any UI changes beyond
the size selector and its price display.
```

The first argument `add-drink-size-selection` is the **change name**. It becomes a directory name under `openspec/changes/`, so use a descriptive slug: lowercase, hyphens instead of spaces, no special characters. Think of it like a git branch name. It should read clearly in the file system and in `git status`.

Everything after the slug is **intent**: a short description of what you want the feature to do, followed by explicit out-of-scope items. Without this, the agent infers intent from the codebase alone, and the codebase hints at more features than this one change should deliver. Feeding intent and scope boundaries up front keeps the proposal focused on what *you* asked for.

OpenSpec will create a change folder and generate four artifacts. Watch the output as your agent works through the generation.

2. **Examine the generated artifacts.** After the proposal completes, look at what was created. In your **terminal**, run:

```
ls openspec/changes/add-drink-size-selection/
```

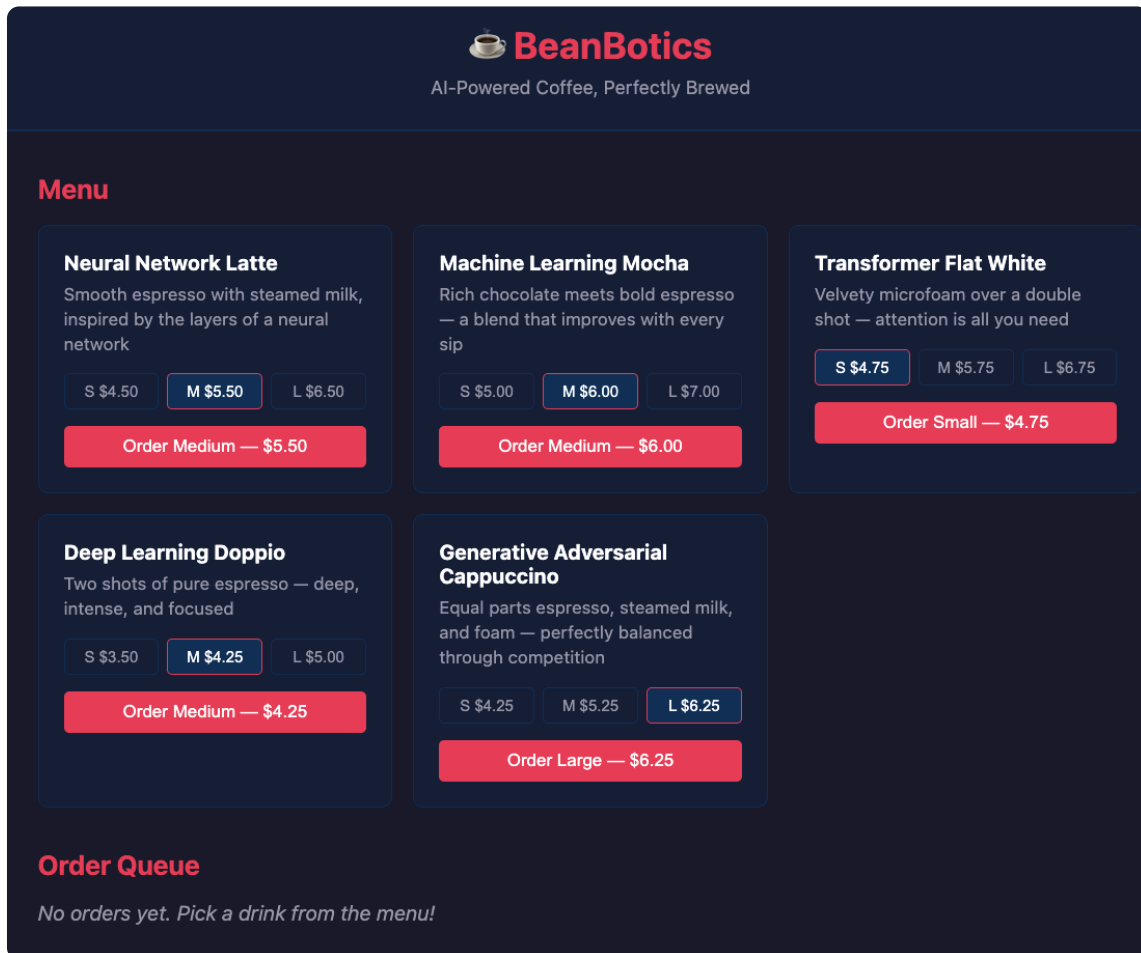
You should see:

- `proposal.md` . Why this change is being made, what is in scope, what is out of scope, and any risks
 - `specs/` . Delta specifications describing what changes in terms of ADDED, MODIFIED, and REMOVED requirements
 - `design.md` . Technical approach: what components change, how, and why
 - `tasks.md` . An ordered checklist of implementation steps
- Open each file and read through it. Ask yourself:
- **proposal.md**: Does the scope match what you intended? Is anything included that should not be?
 - **specs/**: Are the scenarios concrete? Do they describe *behavior* (what the user sees) rather than *implementation* (how the code works)?
 - **design.md**: Does the technical approach make sense given what you learned during exploration?
 - **tasks.md**: Are the tasks small enough to verify individually?

❗ Important

You do not need to edit anything in this first cycle. The goal is to see what OpenSpec generates and develop an eye for evaluating artifacts. In the next lab, you will practice reviewing and refining these artifacts.

3. **Apply the change.** Now tell the AI agent to implement the feature based on the specifications. In your agent's chat panel, invoke the **apply** command (see **Command syntax** above).
Watch as your agent works through the tasks in `tasks.md`. It reads the specifications, design, and task checklist, then implements each step.
4. **Verify the feature.** Once the apply step completes, check the running application in your browser. Refresh the page at <http://localhost:8000>. Your UI should now look something like this:



You should see:

- Each menu card now shows **size options** (small, medium, large) with their respective prices
- Customers can **select a size** before ordering
- The order queue shows the **selected size** in the order details

- Ordering still works correctly. Place an order with a non-medium size and verify the price is correct

If the server needs restarting after the code changes, stop it with

`Ctrl + C` and run

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

 again.

Warning

If something does not work correctly, do not fix it manually. This is a learning opportunity. Note what went wrong. You will learn debugging strategies in a later lab.

5. **Archive the change.** Once you have verified the feature works, archive it. In your agent's chat panel, invoke the **archive** command (see **Command syntax** above).

This does two things:

- Merges delta specs into the main specifications.** Check `openspec/specs/`: it now contains specification files describing the system's current behavior, including the size selection feature you just added
- Moves the change to the archive.** Check `openspec/changes/archive/`: the change folder is preserved with a timestamp for audit purposes

6. **Review the results.** Examine the final state. In your **terminal**, run:

```
# The source of truth -- now describes the current system
ls openspec/specs/

# The archive -- your completed change with timestamp
ls openspec/changes/archive/

# The active changes directory -- should be empty
ls openspec/changes/
```

You have just completed a full propose-apply-archive cycle. The specifications describe what the system does. The archive records what changed and why. The code implements the behavior.

1.8. Reflection Questions

Take a few minutes to consider the following questions. These are not graded. They are meant to deepen your understanding.

1. **How did the specifications guide implementation?** Think about what the AI agent did during the apply step. Did it follow the tasks? Did the specs constrain what it built?
2. **What would have happened without specs?** If you had simply prompted your agent with "add size selection to the ordering UI," how might the result have differed? Would it have made different scope decisions?
3. **What is in `openspec/specs/` now?** Read through the generated specification file. Does it accurately describe the current system behavior? Is there anything missing or inaccurate?
4. **When would this workflow be overkill?** Think about the size of this feature. Was the propose-apply-archive overhead worth it for drink size selection? For what size of change does the overhead start paying off?
5. **How did context clearing help?** Did you notice a difference in the quality of the proposed artifacts after clearing the exploration context?

1.9. Common Issues

"OpenSpec commands not appearing in my agent"

Reload your editor after running `openspec init`. Most agents load their command files when the editor (or the agent extension) starts; until it reloads, the new commands will not appear. In VS Code-based agents, use `Ctrl + Shift + P` → **Developer: Reload Window**. Standalone-IDE agents (Cursor, Windsurf, Kiro) typically pick up new commands on restart.

"Server not starting. Port already in use"

Another process is using port 8000. The most common cause is a previous lab's server still running. Find the terminal where `uv run uvicorn ...` is running and press `Ctrl + C`. If you cannot locate it, restart your terminal session, or run uvicorn with a different `--port` argument.

"`openspec init` fails with permission error"

You may need to install the OpenSpec npm package with elevated permissions, or configure npm to use a different global directory. See the setup guide for details.

"Apply step produces errors or incomplete code"

This is expected in early learning. Common causes:

- The AI agent's context window is cluttered. Try clearing context and invoking the **apply** command again
- The specs are ambiguous. Review the spec files for unclear language
- A task is too broad. The AI tried to do too much in one step

Do not fix code manually in this lab. Note what happened and move on. Debugging strategies are covered in a later lab.

"Archive fails with validation error"

Run `openspec validate` to check for structural issues. The most common cause is incomplete tasks in `tasks.md`: all tasks should be checked off before archiving.

Tip

Your work in `~/LabFiles/foundations-first-cycle/beanbotics/` is preserved after the lab. If you want to restart the lab from the clean starting state, re-extract the course materials ZIP (or copy the pristine folder from a fresh extract).

1.10. Challenge Exercise (Optional)

If you finish early, try this additional exercise.

1.10.1. Propose a Second Feature

Think of another small feature that would improve BeanBotics. Some ideas:

- Display the order timestamp in the order queue
- Add a "popular" badge to drinks with the highest price tier
- Show a confirmation message after placing an order

Run through the full cycle again: propose, review the artifacts, apply, verify, archive. This time, pay closer attention to the quality of the generated specs. Are the scenarios concrete and testable?

Note

Whatever you build here will be part of your project state, but it will not carry forward to the next lab. The next lab starts from a known checkpoint state.

1.11. Context Window Management Tips

These tips go beyond the basics covered in the lecture. You will develop your own feel for context management as you work through the course, but here are two common questions.

1.11.1. How Do I Know When to Clear My Context Window?

There is no hard rule. Watch for these signals:

- **Quality drops.** The agent starts giving vague, repetitive, or off-target responses. It may "forget" things you told it earlier in the conversation or produce code that contradicts earlier decisions.
- **The conversation is long.** If you have been working in the same session for a while. Especially with lots of exploration, debugging, and back-and-forth. The context window is filling up with noise that dilutes the signal.
- **You are switching tasks.** Moving from exploration to specification work, or from one feature to another, is a natural breakpoint. Old context rarely helps and often hurts.

Most agents expose some form of session usage indicator. A token count, a context-fullness meter, or a `/cost`-style command. Find your agent's equivalent and check it periodically; the cache or context read size gives you a rough sense of how full the window is. Your agent cannot check this on its own. You need to look.

Tip

When in doubt, clear. The cost of a fresh start is low (a few seconds). The cost of a cluttered context is subtle and cumulative. You may not notice the

quality degradation until you compare the output to what a fresh session produces.

1.11.2. How Do I Transition Without Losing Important Context?

Before clearing your context or starting a new session, ask the agent to write a **handoff document**: a short summary of what it learned, what decisions were made, and what is still open. Save this to a file in your project.

When you start the new session, point the agent at that file. It picks up where the previous session left off, but with a clean context window free of conversational noise.

```
Write a handoff document summarizing what we've learned, decisions made, and open questions. Save it to docs/handoff.md.
```

This pattern works for any context transition: clearing within a session, switching between sessions, or handing off to a teammate. The specifications themselves serve this purpose for *what the system does*, but a handoff document captures the *working context*: what was tried, what was ruled out, and why.

Exploring, Proposing & Specifying

LAB 2

-
- ✓ Use the **explore** command to investigate an existing codebase and discover missing functionality
 - ✓ Practice context hygiene by clearing context between exploration and specification work
 - ✓ Propose a feature informed by exploration findings using the **propose** command
 - ✓ Review and refine AI-generated artifacts: proposal, specs, design, and tasks
 - ✓ Complete a full explore-propose-apply-archive cycle with a refined specification

2.1. Prerequisites

Before starting this lab, ensure you have:

- Completed the Foundations & First Cycle lab (or have the checkpoint files)
- **Python 3.10+** with `pip`
- **Node.js 18+** with `npm`
- **VS Code** (or your preferred editor) with one of the seven supported AI coding agents installed and signed in (see the **Command syntax** block below)
- **OpenSpec CLI** installed (`openspec --version` to verify)

Note

Command syntax depends on your agent

Throughout this course, when prose says "invoke the **propose** command," substitute the command name into the pattern for your agent:

Agent	Example: propose
Claude Code	<code>/opsx:propose</code>
Cursor, Windsurf, GitHub Copilot, Google Antigravity	<code>/opsx-propose</code>
Amazon Q, AWS Kiro	<code>@opsx-propose</code>

Apply the same pattern to every other command (`apply` , `archive` , `continue` , ...).

2.2. Introduction

In the previous lab, the feature scope was handed to you: "add drink size selection." You followed the propose-apply-archive workflow, and it worked. In real development, however, features rarely arrive pre-scoped. More often, you start with a vague idea: "something is missing" or "this could be better."

This lab teaches the **explore-first pattern**. You will investigate the BeanBotics codebase, discover existing infrastructure that is not being used, and use that understanding to write a well-scoped proposal. Then (this is the core exercise) you will **review and refine** the AI-generated artifacts before applying them.

The feature you will build is an **order status lifecycle with a visual queue board**. Right now, orders are either "pending" or "cancelled." But the data model already defines a richer set of statuses that no one has wired up yet. You will discover this during exploration, propose activating it, refine the specs, and implement the result.

2.3. Exercise 1: Open This Lab's BeanBotics Folder

The starting state for this lab is already in place. This is BeanBotics with size selection, OpenSpec initialized, and one archived change. The completed state from a prior lab.

1. **Open this lab's BeanBotics folder in VS Code.** The course materials include a starter application at `~/LabFiles/exploring-proposing-specifying/beanbotics/`. Open that folder as its own VS Code workspace:

```
code ~/LabFiles/exploring-proposing-specifying/beanbotics
```

If VS Code is already open on a different folder, open a new window (File → New Window) and open this folder in the new window, or close the current window and reopen with the command above.

⚠ Important

For the rest of this lab, **all work happens in this editor window** with `beanbotics` as the project root. Your agent and its OpenSpec commands only work when the editor is opened to the directory where OpenSpec is initialized. The commands are already available. This lab's folder ships agent-config directories for all seven supported agents with the OpenSpec commands pre-installed.

2. **Set up the Python environment.** Open a terminal inside the VS Code window (`Ctrl + ``) and run:

```
uv venv
uv pip install -r requirements.txt
```

3. **Clean up from any prior session.** If port 8000 is in use from a previous lab session, stop that server first. The most common cause is the previous lab's `uv run uvicorn ...` still running in another terminal. Find that terminal and press `Ctrl + C`. If you cannot locate the terminal, restart your terminal session.

4. **Run the app.**

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

Verify the app is running at <http://localhost:8000>. You should see the menu with size selection from a prior lab.

2.4. Exercise 2: Explore the Order System

In the previous lab, you explored with freeform prompts. This time, you will use the **explore** command. OpenSpec's dedicated exploration mode. Explore mode puts the AI agent into thinking-partner mode: it investigates, asks clarifying questions, and helps you understand the codebase without creating any artifacts.

1. **Start exploration.** Open your agent's chat panel and invoke the **explore** command (substitute the syntax for your agent from the **Command syntax** block above). When the agent enters explore mode, ask it to investigate the order system:

```
Look at the Order model in the backend. What status values does it define?
How are those statuses actually used in the codebase? Is anything defined but not implemented?
```

2. **Follow the trail.** The agent should discover something interesting. The `Order` model in `backend/models.py` defines a `status` field with a comment listing five values:

- `pending`
- `preparing`
- `ready`

- `completed`
- `cancelled`

But if you look at the actual code in `backend/services/orders.py`:

- `place_order()` always creates orders with status `"pending"`
- `cancel_order()` changes status to `"cancelled"`
- **No code anywhere transitions orders through preparing, ready, or completed**

This is the gap: there is infrastructure designed for a full order lifecycle, but only two of the five statuses are used. Three statuses exist in the model but have no code path to reach them.

3. **Think through the solution.** Continue the exploration by asking the agent to think through what a full lifecycle would look like:

```
If we wanted to activate the full order lifecycle -- pending
through
preparing, ready, and completed -- what would need to change?
Think
about both the backend (API, validation) and the frontend (how
would
the order queue display different statuses?).
```

The agent will analyze the codebase and present options. Because AI responses are non-deterministic, your agent's analysis may be organized differently from your instructor's or your neighbor's. Different labels, different option numbering, different emphasis. That is expected. The underlying **design questions** are the same regardless of how the agent frames them.

As you read the agent's response, watch for these decision points:

- **Who triggers status transitions?** The agent may suggest a separate staff/barista view, automatic timers, or inline "advance" buttons on each order. These are different approaches to the same question: how does an order move from pending to preparing to ready to completed?
- **When can an order be cancelled?** Currently only pending orders can be cancelled. Should that extend to orders that are already being prepared?
- **What happens to completed orders?** Do they disappear from the list? Stay visible in a separate group? Accumulate indefinitely?

- **API design:** One endpoint for all transitions, or separate endpoints per action? How are invalid transitions rejected?
You do not need to answer these questions during exploration. Just make sure you understand them. You will make your decisions in the next step.

4. **Draft a proposal summary.** You now have a clear understanding of the scope and the design decisions involved. Before clearing context, capture that understanding while the agent still has it. And **bake in your decisions** so the draft is not open-ended.

Use a prompt like this, with your decisions included:

```
Good analysis. Let's capture this as a draft proposal summary.  
Here are my decisions on the open questions:
```

- ```
- Advance buttons inline on each order in the existing UI (no
 separate
 staff view, no automatic timers)
- Cancel from pending only (keep the current restriction)
- Completed orders stay visible in a "Completed" group on the
 queue board
- Single endpoint for all status transitions with a transition
 map
 that validates and rejects invalid moves
```

```
Write a draft proposal summary with these decisions baked in.
Include: the problem, proposed solution, scope, out of scope,
and risks.
Save it to docs/explore-status-lifecycle.md.
```

 **Note**

The decisions above keep the feature focused and achievable within lab time. Inline advance buttons avoid the complexity of a separate staff view or background timers. Restricting cancellation to pending orders keeps the transition logic simple. Showing completed orders in a group makes the full lifecycle visible.

The agent will produce a concise summary incorporating your decisions. Review it quickly and adjust anything that does not match your intent.

This draft is a **bridge document**. It captures the agent's findings and your decisions in a durable file that survives context clearing. Without it, the next session starts cold and the **propose** step has to rediscover everything, or worse, make different decisions than you intended.

5. **Clear your context.** With the draft saved to a file, reset your agent's context so the next exercise starts clean. In Claude Code, type `/clear`; in other agents, open a new chat (or use the equivalent context-reset action).

 **Note**

This is the **context hygiene practice** from the lecture. You explored in one context and will propose in a fresh one. Your understanding persists in the draft file and in your head. The conversation noise gets left behind.

## 2.5. Exercise 3: Propose the Order Status Lifecycle

With a fresh context window and a clear understanding of the scope, you are ready to propose.

1. **Run the proposal.** Point the agent at your draft before proposing. This gives it the findings from exploration without the conversation noise. Ask your agent to read `docs/explore-status-lifecycle.md` for context, then invoke the **propose** command (using your agent's syntax) with the change name `add-order-status-lifecycle`.

Watch as the AI agent generates the four artifacts. Because it has both the draft summary and `openspec/config.yaml`, it produces artifacts informed by your exploration without needing to rediscover anything.

2. **Quick orientation.** Before diving into a detailed review, get a quick sense of what was generated. In your terminal:

```
ls openspec/changes/add-order-status-lifecycle/
```

You should see:

- `proposal.md`
- `specs/`
- `design.md`
- `tasks.md`

In the previous lab, you glanced at these artifacts and moved on. This time, you will read each one carefully and make improvements.

## 2.6. Exercise 4: Review and Refine the Artifacts

This is the core exercise of this lab. The AI-generated artifacts are a starting point, not a final product. Your job is to evaluate each artifact against the criteria from the lecture and make edits where needed.

Open each file in VS Code and review it using the guidance below. **Edit the files directly:** these are plain Markdown files, and the AI agent will use whatever you write when it implements the change.

1. **Review the proposal.** Open `openspec/changes/add-order-status-lifecycle/proposal.md` and ask:
  - **Does the scope match your intent?** You want forward-only status transitions and a visual status board. Did the AI add anything beyond that. Order history, analytics, notifications, time estimates? If so, add those to the "out of scope" section.
  - **Is the problem statement clear?** It should say something like: the order model defines five statuses but only two are used. The application has no way to advance orders through the lifecycle.
  - **Are risks identified?** For example: the current UI shows a flat order list; changing it to a grouped display changes the visual structure significantly.  
Make edits as needed. If the proposal includes features you did not intend, remove them and add them to "out of scope." If the problem statement is vague, sharpen it.

2. **Review the specifications.** Open the spec files in `openspec/changes/add-order-status-lifecycle/specs/`. This is the most important artifact to review. Look for:

### **Are requirements using RFC 2119 keywords?**

If you see a requirement like "the system updates the order status," change it to use precise language: "the system **MUST** update the order status" or "the system **SHOULD** display a confirmation."

### **Are scenarios concrete?**

Compare:

```
Weak -- vague outcome
Given an order exists
When the status is advanced
Then the order should update correctly

Strong -- concrete values and observable behavior
Given an order with status "pending"
When the barista advances the order
Then the order status changes to "preparing"
 and the order queue displays the order under the "Preparing"
 group
```

If you find vague scenarios, rewrite them with concrete values.

### Is there a scenario for invalid transitions?

The AI may have generated only happy-path scenarios. Check whether these edge cases are covered:

- Advancing a cancelled order (should be rejected)
- Advancing a completed order (should be rejected. It is already terminal)
- Cancelling an order that is no longer pending (should be rejected)

If any of these are missing, **add them**. This is where your understanding from the exploration step pays off. An example to add if missing:

```
Scenario: Cannot advance a cancelled order
- **Given** an order with status "cancelled"
- **When** a request is made to advance the order
- **Then** the system rejects the request
 and the order status remains "cancelled"
```

### Are there implementation details where behavioral requirements belong?

More detail is better, but the detail should describe *what* the system does, not *how* the code is structured. If you see references to specific CSS classes, variable names, or function signatures, replace them with behavioral language. For example:



- *Implementation detail*: "Use a `status_transitions` dictionary to validate transitions"
  - *Behavioral requirement*: "The system MUST enforce forward-only transitions and reject invalid transition requests"
- Both are specific. The difference: the behavioral version still holds if someone refactors the internals. The implementation version breaks.

Here is a real example the author encountered when testing this lab. The AI generated this requirement for status badges:

```
Badge styles SHALL be implemented as CSS classes:
`.status-pending`, `.status-preparing`, `.status-ready`,
`.status-completed`, `.status-cancelled`.
```

This dictates *how* the badge is built (specific CSS class names) rather than *what* the user sees. Replace it with a behavioral requirement:

```
Each status MUST be visually distinct. The badge color
MUST clearly differentiate pending, preparing, ready,
completed, and cancelled orders.
```

The AI is free to use CSS classes, inline styles, data attributes, or any other technique. The spec only cares that the user can tell statuses apart at a glance. Add as many edge cases and constraints as you want. Just keep them at the behavior level.

3. **Review the design.** Open `openspec/changes/add-order-status-lifecycle/design.md` and check:

- **Does the technical approach fit the existing codebase?** The backend uses a service layer pattern (routes in `app.py` delegate to service classes). The design should follow this pattern.
- **Is the API design sensible?** Check that the design includes an endpoint for status transitions and validation of transition rules.
- **Does the frontend approach make sense?** The existing frontend uses vanilla JavaScript with `fetch()` calls. The design should not introduce new frameworks or libraries.

Notice that `design.md` contains the kind of detail you *removed* from the specs. This is intentional. Specs and design have different jobs:

- **Specs** describe *what* the system does: "The system **MUST** enforce forward-only transitions and reject invalid transition requests."
- **Design** describes *how* to build it: "Define a transition map as a dictionary in `models.py` alongside the enum. Declarative, co-located with the data model, testable in isolation."

Same feature, different layers. If someone replaced the dictionary with a state machine library, every scenario in the spec would still pass. The design doc is *where* implementation choices like data structures, file placement, API verb selection, and architectural patterns belong. This is the complement to the rule in Step 2: strip implementation details from specs, but expect to find them in the design.

4. **Review the tasks.** Open `openspec/changes/add-order-status-lifecycle/tasks.md` and check:

- **Are tasks small and independently verifiable?** A task like "implement the order status feature" is too broad. It should be broken into steps: add the status transition logic, add the API endpoint, update the frontend display, add the advance action.
- **Are dependencies ordered correctly?** Backend changes should come before frontend changes that depend on new API endpoints.
- **Is each task clear enough that you could verify it without reading the code?** For example: "After this task, `PUT /api/orders/1/status` with `{"status": "preparing"}` returns 200 for a pending order and 400 for a cancelled order."

#### Tip

Reviewing and editing artifacts is a skill that develops over the course. You are not expected to catch everything on the first pass. The goal is to build the habit of reading critically rather than accepting defaults.

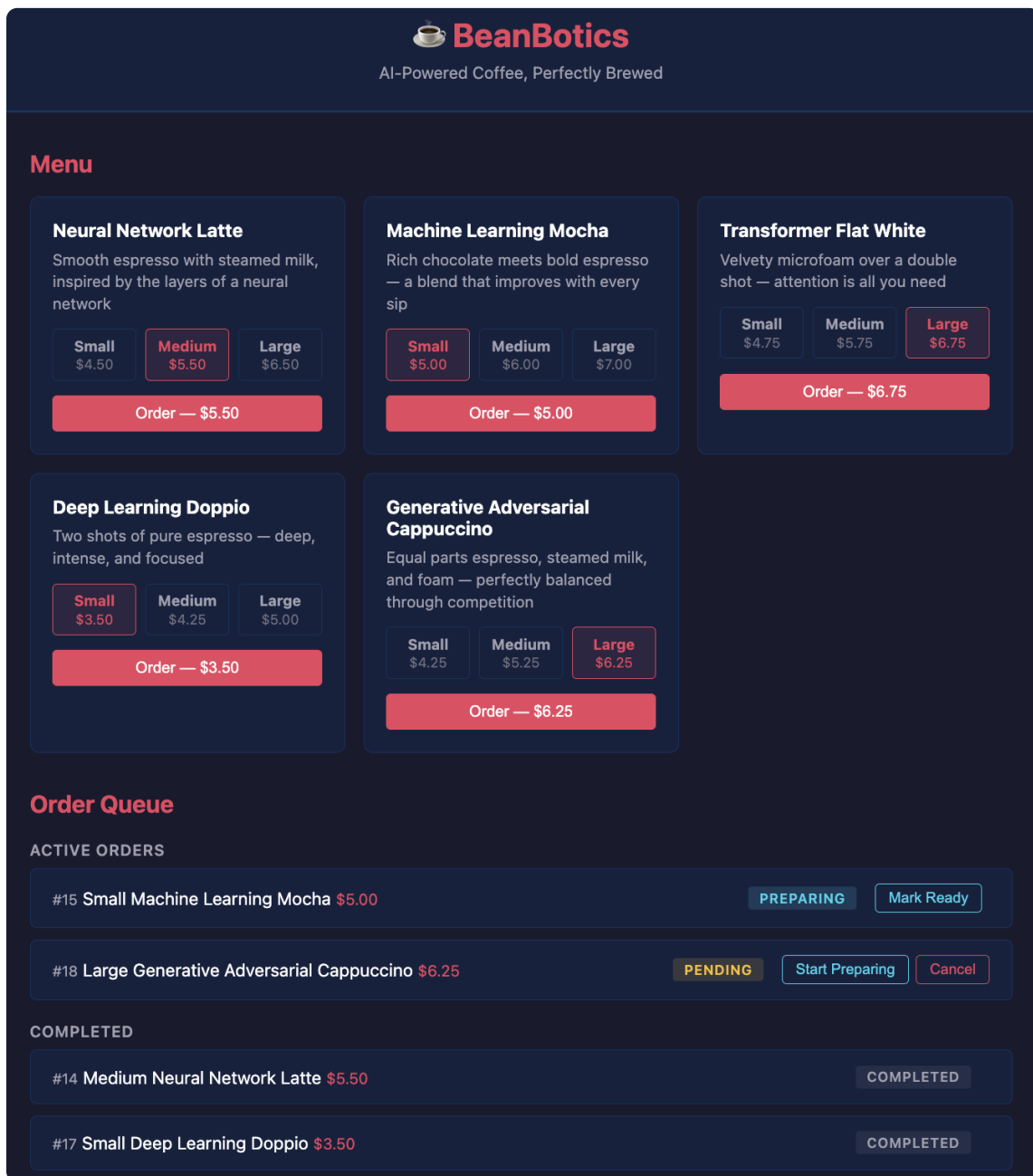
## 2.7. Exercise 5: Apply, Verify, and Archive

1. **Apply the refined specs.** With your refined artifacts in place, invoke the **apply** command (using your agent's syntax) to tell the AI agent to implement the change.

The agent reads your edited specs, design, and tasks. Watch how it works through the task checklist. Because you refined the specs, the

implementation should align more closely with your intent than if you had accepted the defaults.

2. **Verify the feature.** Refresh the application in your browser at <http://localhost:8000>. Place a few orders, advance some through the lifecycle, and test the new functionality. Your UI should look something like this:



**What to verify:**

- **Status-grouped display:** The order queue should organize orders by status rather than showing a flat list. Look for visual groupings like "Pending," "Preparing," "Ready," and "Completed" with color-coded badges or headers.
- **Advance action:** Each non-terminal order should have a way to advance it to the next status. Click it and watch the order move from one group to the next.
- **Forward-only transitions:** Try to advance a completed order. It should not be possible. Cancel a pending order, then try to advance it. It should be rejected.
- **Cancel restriction:** Only pending orders should be cancellable. Orders that have moved to "preparing" or beyond should not have a cancel option.

The server runs with `--reload`, so backend Python changes are picked up automatically. However, your browser caches frontend files (JavaScript, CSS). If the UI looks unchanged after apply, do a **hard refresh** to bypass the cache: `Ctrl + Shift + R`. If that does not help, stop the server with `Ctrl + C` and run `uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000` again.

 **Warning**

If a **behavioral** requirement is not met. Wrong transition rules, missing validation, incorrect grouping. Think about whether the spec is unclear before asking the AI to fix code. Fix the spec first, then re-apply.

 **Tip**

If the behavior is correct but something **looks wrong**: an ugly button, poor spacing, a color that does not fit the theme. Just tell the coding agent to fix it directly. You do not need OpenSpec for cosmetic fixes. The spec says "display an action to advance each non-terminal order." The AI chose a button and the button works, but it is the wrong shade of green? That is not a spec issue. Ask the agent: "Make the advance button match the style of the existing cancel button." This is the "when SDD is overkill" principle

in practice: if every scenario in the spec still passes, the fix does not need a spec.

3. **Archive the change.** Once the feature is working, invoke the **archive** command (using your agent's syntax).
4. **Review the accumulated specs.** In your terminal, examine the specifications after archiving:

```
ls openspec/specs/
```

You now have spec files covering **two domain areas**: drink size selection (from the previous lab) and order status lifecycle. The archive contains both completed changes with timestamps.

```
ls openspec/changes/archive/
```

These specs are becoming the **source of truth** for the BeanBotics system. If a new developer joined the project, they could read `openspec/specs/` to understand what the system does without reading every line of code.

### Tip

Your work in `~/LabFiles/exploring-proposing-specifying/beanbotics/` is preserved after the lab. If you want to restart the lab from the clean starting state, re-extract the course materials ZIP (or copy the pristine folder from a fresh extract).

## 2.8. Reflection Questions

Take a few minutes to consider these questions.

1. **How did exploration change your proposal?** Compare what you would have proposed without exploring to what you actually proposed after discovering the unused status values. Did exploration narrow or widen the scope?
2. **How useful was the draft proposal summary?** Compare the draft you wrote at the end of exploration to the artifacts OpenSpec generated. Did the draft help the AI produce better artifacts? Was anything in the draft lost or misinterpreted?

3. **What did you change in the artifacts?** Think about the edits you made during the review step. Which artifact needed the most work? What kinds of issues did you find?
4. **How did spec quality affect implementation?** If you tightened a vague scenario or added an edge case, did the AI handle it correctly during apply? If you left something vague, did the AI make assumptions you did not expect?
5. **What would "accept all defaults" have produced?** If you had skipped the review step entirely and gone straight from propose to apply, how might the result have differed?
6. **When is the review step worth the time?** This lab's feature was moderately complex. For a smaller change, would the review step still be worth it? Where is the threshold?

## 2.9. Common Issues

### "Menu stuck on 'Loading menu...' and never populates"

The browser tab is left over from a previous lab. It was open before the current server started, so the menu fetch is pending against a server that no longer exists. Do a hard refresh with `Ctrl + Shift + R`. If that does not fix it, confirm the URL is `http://localhost:8000/` and that `uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000` is still running in the terminal.

### "The explore command does not seem to do anything different"

Explore mode is conversational. It does not create files or artifacts. The agent investigates and discusses, but you will not see new files in `openspec/changes/`. That is intentional. Explore is for understanding; propose is for artifacts.

### "The AI generated specs that are too implementation-specific"

This is common, especially with detailed codebases. The AI reads the code and mirrors its structure in the specs. When you see function names, CSS classes, or variable names in specs, replace them with behavioral descriptions. This is normal and expected. It is why the review step exists.

### "I am not sure what to edit in the artifacts"

Start with the specs and look for scenarios. Ask: "Could I test this scenario without reading the code?" If the answer is no, the scenario needs more concrete values or clearer outcomes. You do not need to rewrite everything. Even one improved scenario is valuable practice.

### "UI looks unchanged after apply"

The server picks up backend (Python) changes automatically via `--reload`, but your browser caches frontend files (JavaScript, CSS). Do a hard refresh with `Ctrl` + `Shift` + `R`. If that does not help, stop the server with `Ctrl` + `C` and restart with `uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000`.

### "Apply produced errors or incorrect behavior"

Check the specs first. A common cause is conflicting requirements or missing edge cases. If the spec says orders "can be" cancelled but does not specify *which* statuses allow cancellation, the AI may make a different assumption than you intended. Fix the spec, clear context, and re-apply.

### "The frontend looks different from what I expected"

The spec describes *behavior* (orders grouped by status, advance action available), not *appearance* (specific colors, layout, animations). Visual differences are acceptable as long as the behavior matches. If a behavioral requirement is not met, that is a spec issue.

### "Archive fails with validation errors"

Run `openspec validate` to identify structural issues. The most common cause is unchecked tasks in `tasks.md`. Ensure all tasks are marked complete before archiving.

## 2.10. Challenge Exercise (Optional)

If you finish early, try one of these additional exercises.

### 2.10.1. Write a Scenario From Scratch

Think of an edge case that is not covered by the specs you just archived. For example:

- What happens when multiple orders are advanced simultaneously?
- What should the queue show when there are no orders in a particular status group?
- Should completed orders disappear after a certain time or accumulate indefinitely?

Write a complete Given/When/Then scenario for your chosen edge case. Do not implement it. Just write the scenario and evaluate whether it is concrete enough for an AI agent to implement correctly.

### 2.10.2. Compare Spec Quality

Open the archived spec from the first lab ( [openspec/changes/archive/](#) : the drink size selection change) and compare it to the order status lifecycle spec you just archived. Consider:

- Which has more concrete scenarios?
- Which better separates behavior from implementation?
- How would you improve the first lab's spec with what you know now?

#### Note

Whatever you build in the challenge exercise is for learning purposes. It will not carry forward to the next lab, which starts from a known checkpoint state.

## 2.11. Context Hygiene Recap

This lab practiced the **draft-and-clear** pattern: explore, draft a proposal summary, clear context, then propose using the draft as input.



The draft proposal summary is the key step. It solves two problems at once:

- **It captures what the agent learned.** Without a draft, clearing context discards the agent's findings. The next session starts cold and the **propose** step has to rediscover everything, or worse, guesses.
- **It filters out noise.** A two-paragraph summary preserves the signal (what to build, what to avoid) without the dead ends, corrections, and back-and-forth of an exploration conversation.

When is the draft-and-clear pattern worth it?

- **You explored an unfamiliar area of the codebase.** The exploration was valuable but noisy.
- **The exploration involved dead ends or corrections.** Noise that would confuse the **propose** step.
- **You want artifacts that stand on their own.** The draft ensures spec quality is not diluted by conversation history.

You do *not* always need to clear context. If your exploration was brief and focused, and you are ready to propose in the same session, the overhead may not be worth it. Use judgment. Context hygiene is a skill, not a rigid rule.

# Implementation Patterns & Debugging

**LAB 3**

- 
- ✓ Complete two propose-apply-archive cycles with different levels of review depth
  - ✓ Reference existing specifications when proposing a change that depends on prior work
  - ✓ Observe how the AI agent executes tasks during **apply**
  - ✓ Trace unexpected implementation behavior back to a gap in the specification
  - ✓ Apply the debugging strategy: identify the spec gap, add the missing requirement, re-apply

## 3.1. Prerequisites

Before starting this lab, ensure you have:

- Completed the Exploring, Proposing & Specifying lab (or have the checkpoint files)
- **Python 3.10+** with `pip`
- **Node.js 18+** with `npm`
- **VS Code** (or your preferred editor) with an AI coding agent installed and signed in
- **OpenSpec CLI** installed ( `openspec --version` to verify)

### Note

Command syntax depends on your agent

Throughout this course, when prose says "invoke the **propose** command," substitute the command name into the pattern for your agent:

| Agent                                                | Example: <b>propose</b>    |
|------------------------------------------------------|----------------------------|
| Claude Code                                          | <code>/opsx:propose</code> |
| Cursor, Windsurf, GitHub Copilot, Google Antigravity | <code>/opsx-propose</code> |
| Amazon Q, AWS Kiro                                   | <code>@opsx-propose</code> |

Apply the same pattern to every other command ( `apply` , `archive` , `continue` , ...).

## 3.2. Introduction

In the previous labs, you built one feature per lab. You learned the core workflow (propose, apply, archive), how to explore before proposing, and how to review and refine artifacts. Each feature stood on its own. Size selection and the order lifecycle were independent changes.

This lab changes that. You will build **two features that depend on each other**:

1. **Drink customizations:** customers can add extras to their drinks: extra shot (\$0.75), milk alternatives like oat, almond, or soy (\$0.60), and whipped cream (+\$0.50).
2. **Order confirmation and receipt view:** before placing an order, customers see a confirmation step. After an order is completed, they can view a receipt with a line-item breakdown: base price, each customization, tax, and total.

The dependency: the receipt must show customization details and prices from the first feature. This is the first time a change you build depends on a change you just finished.

Along the way, you will practice two new skills:

- **Adjusting your review depth.** Not every change needs the same scrutiny. Change 1 is well-scoped. A quick scan of the artifacts is enough. Change 2 has a dependency and a subtle spec ambiguity that requires closer attention.
- **Debugging spec ambiguity.** When the implementation does not match your expectations, the root cause is often a gap in the spec, not a bug in the code. You will experience this firsthand and learn to trace the problem back to its source.

### 3.3. Exercise 1: Set Up Your Working Copy

This is the completed state from a previous lab. BeanBotics with size selection, the order status lifecycle, and two archived changes.

1. **Open this lab's BeanBotics folder in VS Code.** The starter state for this lab is at `~/LabFiles/implementation-patterns-debugging/beanbotics/`. Open it as its own VS Code workspace:

```
code ~/LabFiles/implementation-patterns-debugging/beanbotics
```

If VS Code is already open on a different folder, open a new window (File → New Window) and open this folder in the new window, or close the current window and reopen with the command above.

❗ Important

For the rest of this lab, **all work happens in this VS Code window** with `beanbotics` as the project root. Your agent's OpenSpec commands are already available. The starter state includes the agent-config directories from the earlier labs.

2. **Set up the Python environment.** Open a terminal inside the VS Code window ( `Ctrl` + ``` ) and run:

```
uv venv
uv pip install -r requirements.txt
```

3. **Clean up from any previous lab session.** If you still have a server running from an earlier lab, stop it now. Press `Ctrl` + `C` in the terminal where it is running. If you cannot locate that terminal, restart your terminal session to release the port.

4. **Run your app.**

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

Verify the app is running at <http://localhost:8000>. You should see the menu with size selection and the status-grouped order queue from the earlier labs.

5. **Review your starting state.** Before building new features, take a quick look at what you are starting with. In your terminal:

```
ls openspec/specs/
```

You should see three spec files from the two features built in previous labs: `drink-size-selection`, `order-status-lifecycle`, and `order-status-ui`. These are the behavioral contracts for the system as it exists now.

```
ls openspec/changes/archive/
```

Two archived changes with timestamps. The complete history of what has been built so far.

This accumulated spec suite is important. In the next exercise, you will build on it, and in Exercise 3, you will reference it explicitly when proposing a dependent change.

### 3.4. Exercise 2: Drink Customizations. The Fast Cycle

This exercise is a complete propose-apply-archive cycle, but at a faster pace than previous labs. The feature is well-scoped and the scope is clear from the start. You do not need to explore first, and you do not need to do a deep artifact review.

The goal: practice adjusting your review depth to match the complexity of the change. Not every feature needs the same level of scrutiny. Drink customizations is a straightforward addition. Save your detailed review energy for Exercise 3.

1. **Propose drink customizations.** Open your agent's chat panel and invoke the **propose** command (see **Command syntax** above) with the change name `add-drink-customizations` and the following prompt body:

```
Customers should be able to add optional extras when ordering any drink:
```

- Extra espresso shot: +\$0.75
- Milk alternative (oat, almond, or soy): +\$0.60
- Whipped cream: +\$0.50

```
Customization options should appear on each menu card. The order total should update dynamically as customizations are selected. The order queue and status board should display any customizations on each order.
```

```
Out of scope: per-drink customization restrictions (all options available for all drinks), customization inventory tracking, recurring/saved customization preferences.
```

#### Note

This proposal includes specific prices, a clear list of options, scope boundaries, and explicit out-of-scope items. Because the intent is precise, the AI generates better artifacts with less correction needed.

2. **Quick scan of artifacts.** Open the generated artifacts in VS Code. This time, do a **quick scan** rather than the deep review you practiced in the previous lab:

- **proposal.md** : Does the scope match your intent? Did the AI add features you did not ask for? (Skim in under a minute.)
- **specs/** : Are customization prices correct (\$0.75, \$0.60, \$0.50)? Do the scenarios cover adding customizations to an order and seeing them in the queue? (Skim the scenarios for obvious gaps.)
- **design.md** : Does the approach fit the existing codebase? (Quick check. Not a deep review.)
- **tasks.md** : Are backend tasks before frontend tasks? (Glance at the ordering.)

If anything is clearly wrong (wrong prices, scope creep, obviously missing scenarios), fix it. Otherwise, move on. The goal is to build the habit of matching review depth to risk. This change is well-scoped with clear requirements, so a light review is appropriate.

#### Note

During lab testing, the course author found that **design.md** contained several instances of implementation-level detail that belongs in specs or tasks, not design. For example, the AI generated a decisions section that named specific Python variables and type signatures: *"Define CUSTOMIZATIONS: dict[str, float] in models.py mapping option IDs to prices."* The design-level version of that decision is: *"The set of options is small, universal across all drinks, and unlikely to change frequently. Defining them as a constant in the backend keeps them co-located with the data model."* The principle: design documents capture **decisions and rationale** (why X over Y), not variable names, type signatures, or file paths. If you spot this in your artifacts, clean it up, but do not let it block you from moving forward. See the extra material at the end of this lab for more detail.

3. **Apply and observe.** Before running apply, keep these questions in mind as you watch the agent work:

- Does the agent work through tasks in the order listed in **tasks.md**?
- When it hits something not covered by the spec, does it ask or assume?
- Does it refer back to the specs or design, or does it just write code?
- Does it check off tasks as it completes them?

Now invoke the **apply** command.

Watch the agent work. You do not need to intervene. Just observe. After it finishes, note your answers to the questions above. You will discuss these observations during the reflection at the end of the lab.

4. **Verify in the browser.** Refresh the application at `http://localhost:8000`. If the UI looks unchanged, do a hard refresh: `Ctrl + Shift + R`.

**What to verify:**

- **Customization options visible:** Each menu card should show options for extra shot, milk alternative, and whipped cream.
- **Dynamic price updates:** Selecting customizations should update the displayed order price. A Large Neural Network Latte (\$6.50) with an extra shot (\$0.75) should show \$7.25.
- **Order placement with customizations:** Place an order with one or more customizations. The order queue should show the customizations on the order card.
- **Customizations on the status board:** Advance a customized order through the lifecycle. The customization details should remain visible at each status.

 **Tip**

If the behavior is correct but something looks rough. Awkward layout, clashing colors, poor spacing. Just tell the agent directly: "Fix the customization layout to match the existing card style." No spec needed for cosmetic fixes.

5. **Archive.** Invoke the **archive** command.

Verify that the customization specs have been merged into

`openspec/specs/`:

```
ls openspec/specs/
```

You should see new spec files covering the customization domain alongside the existing size selection and order lifecycle specs.

6. **Clear context.** Clear your conversation context before starting the next change. Most agents support this through a "new chat" / "clear context"



command or button. Consult your agent's docs for the exact mechanism (for example, Claude Code uses `/clear`).

Change 2 depends on Change 1, but you will handle that dependency explicitly in the next exercise. By pointing the agent at the archived specs, not by carrying forward conversation history.

### 3.5. Exercise 3: Order Confirmation & Receipt. Dependency Handling

This change depends on the customization feature you just built. Receipts must show each customization as a separate line item with its individual price. This is the first time you are proposing a change that explicitly depends on a prior change.

1. **Reference existing specs when proposing.** After archiving Change 1, the customization specs were merged into `openspec/specs/`. The agent can see the **code** from Change 1 (it is in the codebase), but the **specs** capture behavioral contracts. Edge cases, constraints, and pricing rules. That the code alone does not communicate.  
Feed the specs as context when proposing. First, send the following message to your agent:

Read the specs in `openspec/specs/` for context on the current system, especially the customization pricing model.

This change adds two things:

1. An order confirmation step: after selecting a drink and customizations,

the customer sees a "Review Your Order" summary before confirming.

The summary shows the drink name and size, each selected customization

with its price, and the order total. The customer can confirm or go back.

2. A receipt view for completed orders: when an order reaches "completed"

status, it has a receipt showing a line-item breakdown:

- Base price (drink name, size, price)
- Each customization as a separate line item with its price
- Subtotal
- Total including tax

Out of scope: printing receipts, emailing receipts, receipt history page, tax configuration UI.

Then invoke the **propose** command with the change name `add-order-confirmation-receipt`.

#### Note

Notice what the proposal does *not* say: the tax rate, whether tax is applied per item or on the subtotal, and how to round. The phrase "total including tax" is deliberately vague. You will investigate what the AI does with that ambiguity in Exercise 4.

2. **Review artifacts. Focus on the dependency.** This time, review the artifacts with more care than Exercise 2. Pay attention to how the generated specs handle the dependency on customizations:

In `proposal.md`:

- Does it acknowledge the dependency on customizations?

- Is the scope right? The confirmation step and receipt view are distinct features. Make sure neither one was dropped or over-expanded.

**In** `specs/` :

- Do the receipt scenarios reference customization line items? There should be a scenario showing a receipt with at least one customization.
- Is the line-item structure specified? The receipt should break down: base price, customizations (each on its own line with price), subtotal, tax, total.
- Is the tax calculation specified? Look for: the tax rate, what it applies to (subtotal), and how rounding works. **Note what is present and what is missing:** you will need this information in Exercise 4.

**In** `design.md` :

- Does the API design make sense? A receipt likely needs a new endpoint or an extension to the existing order endpoint.
- Does the frontend approach fit the existing vanilla JS codebase?

**In** `tasks.md` :

- Are receipt-related tasks after the confirmation step? (The confirmation step is simpler and does not depend on pricing logic.) Edit any issues you find. In particular, if the tax rate or rounding strategy is not specified in the specs, **leave it that way for now**. You will use that gap as a debugging exercise in Exercise 4.

### 3. **Apply.** Invoke the **apply** command.

Watch the agent work through the tasks. Because the specs reference customization behavior from the archived specs, the agent should produce an implementation that integrates customization line items into the receipt.

### 4. **Verify the confirmation step.** Refresh the application at

`http://localhost:8000` (hard refresh: `Ctrl` + `Shift` + `R` ).

- **Select a drink and customizations.** Click the order button.
- **Confirmation step appears.** You should see a "Review Your Order" summary showing the drink, size, any customizations, and the total price.
- **Confirm or go back.** Confirming should place the order. Going back should return to the menu without placing the order.

### 3.6. Exercise 4: Debugging Spec Ambiguity

Now for the debugging exercise. You will verify the receipt feature with a specific test order and check whether the implementation matches your expectations.

1. **Create a test order.** Place a specific order so you can calculate the expected receipt values by hand:
  - i. Select a **Large Neural Network Latte** (\$6.50)
  - ii. Add an **extra shot** (+\$0.75)
  - iii. Confirm and place the order
2. **Calculate the expected receipt.** Before looking at the receipt, calculate what it should show:

|                                  |        |
|----------------------------------|--------|
| Large Neural Network Latte ..... | \$6.50 |
| Extra espresso shot .....        | \$0.75 |
| Subtotal                         | \$7.25 |
| Tax (8.5%)                       | \$0.62 |
| Total                            | \$7.87 |

#### Note

The tax calculation:  $\$7.25 \times 0.085 = \$0.61625$ , rounded to the nearest cent = \$0.62. Total:  $\$7.25 + \$0.62 = \$7.87$ .

3. **Advance the order and check the receipt.** Advance the test order through the lifecycle: pending → preparing → ready → completed. Once it reaches **completed** status, view the receipt.  
Compare the receipt to your hand calculation. Check:
  - **Are the line items correct?** Base price, customization, subtotal.
  - **Is the tax rate 8.5%?** If the spec did not specify a rate, the AI chose one. What did it choose?
  - **Is the tax amount correct?** Calculate it yourself and compare.
  - **Does the total match?** Even a one-cent difference matters. It reveals a spec gap.
4. **Diagnose any differences.** If the receipt matches your calculation exactly, check whether the spec actually specified all the details. Open the spec files

in `openspec/changes/` (if the change is still active) or review what you saw during the Step 2 review:

- **Did the spec specify the tax rate?** If not, the AI guessed correctly this time, but guessing correctly once is luck, not engineering. A different AI session, a different model, or a different day could produce a different rate.
- **Did the spec specify the rounding strategy?** Per item, or on the final total? If not, the AI made a choice. What was it?
- **Did the spec say "applied to the subtotal"?** Or could the AI have interpreted "including tax" as tax applied per line item?

If the receipt does **not** match your calculation:

- Identify the gap.** What question did the spec leave unanswered? Tax rate? Rounding? Calculation order?
- Check the implementation.** What did the AI decide? Look at the backend code or ask the agent: "What tax rate does the receipt calculation use?"
- Fix the spec.** Open the spec file and add the missing requirement with concrete values. For example:

```
Requirement: Tax calculation
Tax MUST be calculated at a rate of 8.5%, applied to
the order
subtotal (base price plus all customizations). The tax
amount
MUST be rounded to the nearest cent using standard
rounding
(half-up). Tax MUST appear as a separate line item on
the receipt.
```

- Re-apply.** Clear context and invoke the **apply** command to let the agent update the implementation based on the corrected spec. This is the core debugging lesson: **the bug was in the spec, not the code.** The AI implemented exactly what the spec described. If the spec was incomplete, the AI filled the gap with an assumption, and that assumption may or may not match your intent.

- Verify the fix.** If you made spec changes and re-applied, verify the receipt again with the same test order. The numbers should now match your hand calculation exactly.

If they still do not match, you have a **code bug**, not a spec bug. The spec is clear, but the implementation is wrong. Point the agent at the spec and the incorrect code:

```
The receipt spec says tax is 8.5% applied to the subtotal,
rounded
to the nearest cent. But the receipt shows $0.61 instead of
$0.62
for a $7.25 subtotal. Check the tax calculation in the backend
and
fix it to match the spec.
```

6. **Archive.** Once the receipt is correct, invoke the **archive** command.

### 3.7. Exercise 5: Review the Accumulated System

1. **Examine the spec suite.**

```
ls openspec/specs/
```

Your `openspec/specs/` directory now covers **four domain areas**: drink size selection, order status lifecycle, drink customizations, and order confirmation with receipts. The number of files may exceed four. AI agents often split a domain into separate UI and data specs (for example, `drink-customizations` and `drink-customizations-ui`). This is becoming a comprehensive behavioral description of the BeanBotics system.

2. **Examine the archive.**

```
ls openspec/changes/archive/
```

Four archived changes with timestamps. Each one records what changed, why, and how. A complete audit trail of the system's evolution.

3. **Read a spec.** Pick one of the spec files in `openspec/specs/` and read through it. Consider:

- Could a new developer read this spec and understand what the system does without reading the code?
- Are the scenarios concrete enough to verify?
- How has the spec suite grown since the first lab?

The specs are becoming a **living document** of the system's behavior. Each lab adds to it. In the next lab, you will use the expanded OpenSpec workflow to build a more complex feature, and the accumulated specs will provide critical context.

## 3.8. Reflection Questions

Take a few minutes to consider these questions.

1. **How did you adjust your review depth?** Compare how you reviewed artifacts in Exercise 2 (quick scan) vs. Exercise 3 (focused review). What made the difference? Was the quick scan appropriate for Change 1, or did you miss something that mattered?
2. **What did you observe during the apply step?** Did the agent follow the task order? Did it ask or assume when it hit ambiguity? Did it reference the specs or design? How does your answer affect how much detail you put in future specs?
3. **How did referencing existing specs change Change 2?** Did the receipt artifacts reference customization line items? Would they have done so if you had not pointed the agent at `openspec/specs/`?
4. **What did the tax exercise reveal?** Did your spec specify the tax rate and rounding strategy? If not, what did the AI decide? Was the gap obvious before you checked, or did you only notice it when the numbers did not match?
5. **Spec bug vs. code bug:** In your debugging experience, was the issue a spec gap (incomplete requirement) or a code bug (correct spec, wrong implementation)? How did you tell the difference?
6. **How would you approach the next feature differently?** Based on what you learned about dependencies and spec ambiguity, what would you do differently when proposing a feature that touches pricing or calculations?

## 3.9. Common Issues

### "Menu stuck on 'Loading menu...' and never populates"

The browser tab is left over from a previous lab. It was open before the current server started, so the menu fetch is pending against a server that no longer exists. Do a hard refresh with `Ctrl + Shift + R`. If that does not fix it, confirm the URL

is `http://localhost:8000/` and that `uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000` is still running in the terminal.

### "Customization options not appearing on menu cards"

The server picks up backend changes automatically, but the browser caches frontend files. Do a hard refresh with `Ctrl + Shift + R`. If that does not help, stop the server with `Ctrl + C` and restart with

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000.
```

### "Order total does not update when selecting customizations"

Check whether the frontend JavaScript was updated to handle customization price changes. If the spec described dynamic price updates but the implementation does not include them, the task may have been too broad. Ask the agent to look at `tasks.md` and implement the missing subtask.

### "Confirmation step does not appear"

The confirmation step requires frontend changes. If the menu still places orders directly without a review step, the apply may not have completed all tasks. Run `openspec validate` to check, then re-invoke the **apply** command if tasks are incomplete.

### "Receipt does not show customization line items"

This is likely a dependency issue. Check whether the receipt spec references customizations. If the spec says "display the total" but does not mention individual customization line items, the agent may have shown only a summary. Fix the spec to require itemized customizations, then re-apply.

### "Tax amount is off by a cent"

This is the spec ambiguity exercise working as designed. See Exercise 4 for the debugging strategy. The root cause is almost always a missing or imprecise requirement about the tax rate, calculation order, or rounding strategy.

### "Archive fails with validation errors"

Run `openspec validate` to identify structural issues. The most common cause is unchecked tasks in `tasks.md`. Ensure all tasks are marked complete before archiving.



## 3.10. Challenge Exercise (Optional)

If you finish early, try one of these exercises.

### 3.10.1. Manually Add a Tax Scenario

Write a new Given/When/Then scenario for a multi-customization receipt. For example:

```
Scenario: Receipt with multiple customizations
- **Given** a completed order for a Small Machine Learning Mocha
 ($5.00)
 with extra shot (+$0.75) and whipped cream (+$0.50)
- **When** the customer views the receipt
- **Then** the receipt shows:
 - Small Machine Learning Mocha: $5.00
 - Extra espresso shot: $0.75
 - Whipped cream: $0.50
 - Subtotal: $6.25
 - Tax (8.5%): $0.53
 - Total: $6.78
```

Add this scenario directly to the relevant spec file in `openspec/specs/`. This is practice for writing scenarios by hand rather than relying on the AI to generate them. Calculate the tax yourself:  $\$6.25 \times 0.085 = \$0.53125$ , rounded to \$0.53.

### 3.10.2. Compare Your Two Cycles

Reflect on the difference between Exercise 2 (fast cycle) and Exercise 3 (thorough cycle):

- How much time did each take?
- Did the fast cycle produce any issues that a deeper review would have caught?
- Where is the line between "fast enough" and "too fast"?

Write down your personal criteria for when to do a quick scan vs. a deep review. You will refine these criteria over the remaining labs.

 **Note**

Whatever you build in the challenge exercises is for learning purposes. It will not carry forward to the next lab, which starts from a known checkpoint state.

## 3.11. Context Hygiene Recap

This lab practiced a new context hygiene pattern: **clearing between dependent changes**.

In the previous lab, you cleared context between exploration and proposing. Different phases of the same feature. In this lab, you cleared context between two **different features**, then used the archived specs as the bridge.

The pattern:

1. Build and archive Change 1
2. Clear context
3. Point the agent at the relevant specs in `openspec/specs/`
4. Propose Change 2 with the dependency made explicit

This works because specs persist in files. When you archive, the behavioral contracts merge into `openspec/specs/` and are available to any future session. The agent does not need to remember what it built. It reads the spec.

Compare this to the alternative: keeping context from Change 1 and proposing Change 2 in the same session. That works too. The agent remembers what it just built, but the context window now carries all the noise from Change 1: exploration, failed approaches, cosmetic fixes, debugging side conversations. That noise can degrade the quality of Change 2's artifacts.

The choice depends on the situation:

- **Clear between changes** when Change 1 involved significant debugging or back-and-forth. The noise outweighs the continuity.
- **Keep context** when Change 1 was clean and quick, and Change 2 is closely related. The continuity helps.

Use judgment. Context hygiene is about managing signal-to-noise, not following a rigid rule.

## 3.12. Extra Material: Design Doc Abstraction Level

During lab testing, the AI-generated `design.md` for the drink customizations change contained implementation details that belong in specs or tasks. Here are the examples that were found and corrected:

### Context section. Before:

```
The Order dataclass stores items: List[str] (a display name like
"Medium Neural Network Latte") and total_price: float (the size
price). The POST /api/orders endpoint accepts
{ "item_id": "...", "size": "..." } only.
```

### Context section. After:

```
BeanBotics orders currently track a drink name, size, and price.
There is no concept of add-ons or modifiers – every order is the
base drink at the selected size.
```

### Decisions section. Before:

```
Define CUSTOMIZATIONS: dict[str, float] in models.py mapping option
IDs to prices. This keeps options co-located with the data model
and avoids a separate data file for three items.
```

### Decisions section. After:

```
The set of options is small (five items), universal across all
drinks, and unlikely to change frequently. Defining them as a
constant in the backend keeps them co-located with the data model
and avoids a separate data file.
```

### Risks section. Before:

```
POST /api/orders gains a new field, but it defaults to [],
so existing callers without the field still work.
```

**Risks section. After:**

The order API contract changes, but the new field is optional so existing callers are unaffected.

**The principle:** Design documents capture decisions and rationale. Why X over Y. Specs define the contract. What the system must do, with testable scenarios. Tasks list implementation steps. Where to make changes. When a design doc includes variable names, type signatures, file paths, exact API bodies, or default values, it is leaking spec-level or task-level detail into the wrong artifact.

 **Tip**

Your work in `~/LabFiles/implementation-patterns-debugging/beanbotics/` is preserved after the lab. If you want to restart the lab from the clean starting state, re-extract the course materials ZIP (or copy the pristine folder from a fresh extract).

# Expanded Workflow & Spec Lifecycle

## LAB 4

- 
- ✓ Switch from the core profile to the expanded profile and use the expanded command set
  - ✓ Use the **new** and **continue** commands to build artifacts incrementally with review gates
  - ✓ Apply targeted review questions at each artifact stage to catch issues before they propagate
  - ✓ Use the **verify** command to validate implementation against specifications
  - ✓ Identify and fix a spec gap found during verification (customization COGS)
  - ✓ Examine how the spec suite evolves through the archive process

## 4.1. Prerequisites

Before starting this lab, ensure you have:

- Completed the Implementation Patterns & Debugging lab (or have the checkpoint files)
- **Python 3.10+** with `pip`
- **Node.js 18+** with `npm`
- **VS Code** (or your preferred editor) with one of the supported AI coding agents installed, signed in, and configured
- **OpenSpec CLI** installed ( `openspec --version` to verify)

### Note

Command syntax depends on your agent

Throughout this course, when prose says "invoke the **propose** command," substitute the command name into the pattern for your agent:

| Agent                                                | Example: <b>propose</b>    |
|------------------------------------------------------|----------------------------|
| Claude Code                                          | <code>/opsx:propose</code> |
| Cursor, Windsurf, GitHub Copilot, Google Antigravity | <code>/opsx-propose</code> |
| Amazon Q, AWS Kiro                                   | <code>@opsx-propose</code> |

Apply the same pattern to every other command ( `apply` , `archive` , `continue` , ...).

## 4.2. Introduction

In the previous labs, you used the core workflow: propose, apply, archive. Each time, the **propose** command generated all four artifacts (proposal, specs, design, and tasks) in a single step. You reviewed the artifacts after they were all generated and made corrections before applying.

That works well for features you can describe in one prompt, but some features have too many interrelated decisions. An ingredient cost model involves ingredient definitions, recipe mappings that vary by drink and size, COGS calculations that must account for customizations, and a live financial dashboard that aggregates across orders. If the proposal captures the wrong scope, the specs, design, and tasks all inherit the mistake. You discover the problem during review, but by then all four artifacts need rework.

This lab introduces the **expanded workflow**. Instead of generating all artifacts at once, you build them one at a time with **review gates** between each step. You review the proposal before specs are generated. You review the specs before the design is generated. Each correction compounds. Downstream artifacts benefit from the reviewed upstream artifacts.

**The feature:** Ingredient cost model with a financial dashboard. Each drink is a recipe that consumes specific quantities of base ingredients. The financial dashboard shows revenue, cost of goods sold (COGS), and gross margin, updating live as orders are placed and completed.

**Why this feature needs the expanded workflow:** The ingredient cost model involves multiple interrelated decisions. Ingredient definitions, recipe mappings per drink and size, COGS calculations that account for customizations, and a live financial view that aggregates across orders. Before you start, consider: could you write a single proposal prompt that captures all of this with enough precision that the AI gets it right on the first pass?

## 4.3. Exercise 1: Set Up Your Working Copy

This is the completed state from the previous lab. BeanBotics with size selection, order status lifecycle, drink customizations, and order confirmation with receipts.

1. **Open this lab's BeanBotics folder in VS Code.** The starter state for this lab is at `~/LabFiles/expanded-workflow-spec-lifecycle/beanbotics/`. Open it as its own VS Code workspace:

```
code ~/LabFiles/expanded-workflow-spec-lifecycle/beanbotics
```

If VS Code is already open on a different folder, open a new window (File → New Window) and open this folder in the new window, or close the current window and reopen with the command above.

 **Important**

All lab work happens in this VS Code window with `beanbotics` as the project root. The OpenSpec commands are already available. The checkpoint includes the agent config directories generated by previous labs (each agent reads its own tree; the others are inert).

2. **Set up the Python environment.** Open a terminal inside the VS Code window ( `Ctrl` + ``` ) and run:

```
uv venv
uv pip install -r requirements.txt
```

3. **Clean up from the previous lab.** If you still have the server running from the previous lab, stop it now. Switch to the terminal where it is running and press `Ctrl` + `C` . If you cannot locate that terminal, restart your terminal session to release the port.

4. **Run the app.**

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

Verify the app is running at `http://localhost:8000` . You should see the full BeanBotics UI: the menu with size selection and customization options, the order confirmation step, the status-grouped order queue, and receipt viewing for completed orders.

5. **Review your starting state.** Examine what you are starting with:

```
ls openspec/specs/
```

You should see seven spec files covering four domain areas from the previous labs: drink size selection, order status lifecycle (two specs), drink customizations (two specs), and order confirmation with receipts (two specs).

```
ls openspec/changes/archive/
```

Four archived changes with timestamps. The complete history of the BeanBotics system so far.



Now look at the commands available in your agent's config directory. The exact path depends on which agent you chose in Lab 1:

| Agent              | Command directory                   |
|--------------------|-------------------------------------|
| Claude Code        | <code>.claude/commands/opsx/</code> |
| Cursor             | <code>.cursor/commands/</code>      |
| Windsurf           | <code>.windsurf/workflows/</code>   |
| GitHub Copilot     | <code>.github/prompts/</code>       |
| Google Antigravity | <code>.agent/workflows/</code>      |
| Amazon Q           | <code>.amazonq/prompts/</code>      |
| AWS Kiro           | <code>.kiro/prompts/</code>         |

List the contents of your agent's directory. For example, Claude Code users would run:

+

```
ls .claude/commands/opsx/
```

+ You should see the **core profile** commands: `propose`, `apply`, `archive`, and `explore` (the file extension varies by agent). In the next exercise, you will switch to the expanded profile, which adds new commands.

## 4.4. Exercise 2: Switch to the Expanded Profile

Before building artifacts, you need the expanded commands. This exercise switches your OpenSpec profile and explains why.

1. **Read the feature requirements.** Before switching profiles, read through the full feature description. Consider whether you could capture all of this in a single **propose** prompt:

**Ingredient cost model:**

- Define base ingredients with unit costs: espresso (\$0.40/shot), milk (\$0.05/oz), chocolate (\$0.08/oz), whipped cream (\$0.25/serving), alternative milk (\$0.15/oz)
- Map each drink and size to ingredient quantities. For example:
  - Neural Network Latte: Small = 1 shot + 8oz milk, Medium = 1 shot + 10oz milk, Large = 2 shots + 12oz milk
  - Machine Learning Mocha: Small = 1 shot + 6oz milk + 2oz chocolate, Medium = 1 shot + 8oz milk + 3oz chocolate, Large = 2 shots + 10oz milk + 4oz chocolate
- Calculate COGS per order from the recipe: sum of (ingredient quantity x unit cost) for each ingredient
- Customizations add to COGS too. An extra shot adds the cost of one more espresso shot

**Financial dashboard:**

- Display alongside the order board: total revenue, total COGS, and gross margin (revenue minus COGS)
- Show a per-order breakdown: order details, revenue (the price the customer paid including tax), COGS, and margin
- Update live as orders are placed and progress through the lifecycle  
Could you write a single prompt that captures all of this precisely? The ingredient quantities, the COGS calculation including customization costs, the dashboard layout and update behavior? Some of it, probably. All of it with the precision needed to avoid rework? That is harder. This is why the expanded workflow exists.

2. **Switch profiles.** In your terminal (not in your agent's chat panel), run:

```
openspec config profile
```

You will see the current profile settings and a menu asking what to configure:

```
Current profile settings
 Delivery: both
 Workflows: 4 selected (custom)
 Delivery = where workflows are installed (skills, commands, or both)
 Workflows = which actions are available (propose, explore, apply, etc.)

? What do you want to configure?
> Delivery and workflows
 Delivery only
 Workflows only
 Keep current settings (exit)

Update install mode and available actions together
↑↓ navigate • ↵ select
```

Select **Workflows only**. This lets you change which actions are available without changing how they are delivered.

Next, you will see a checkbox list of all available workflows. The core profile has four workflows enabled (Propose change, Explore ideas, Apply tasks, Archive change). Use the arrow keys and Space to toggle on the additional expanded workflows:

```
✓ What do you want to configure? Workflows only
? Select workflows to make available:
>[x] Propose change
 [x] Explore ideas
 [] New change
 [] Continue change
 [x] Apply tasks
 [] Fast-forward
 [] Sync specs
 [x] Archive change
 [] Bulk archive
 [] Verify change
 [] Onboard

Create proposal, design, and tasks from a request
Space to toggle, Enter to confirm
```

Enable these four additional workflows:

- **New change**
- **Continue change**
- **Fast-forward**
- **Verify change**

Leave Sync specs, Bulk archive, and Onboard unchecked. This lab does not use them. Press `Enter` to confirm.

When prompted "Apply changes to this project now?", select **Yes**.

OpenSpec updates the project and regenerates each installed agent's command files automatically. You do not need to run

`openspec update` separately.

```
What do you want to configure? Workflows only
✓ Select workflows to make available: Propose change, Explore ideas, New change, Continue change, Apply tasks, Fast-forward, Archive change, Verify change

Config changes:
 workflows: added new, continue, ff, verify
✓ Apply changes to this project now? Yes
Updating 1 tool(s): claude (config sync)
✓ Updated Claude Code
✓ Updated: Claude Code (v1.2.0)
Tools: Claude Code
Restart your IDE for changes to take effect.
Run 'openspec update' in your other projects to apply.
```

3. **Verify new commands.** The new commands should be available in your agent immediately. Verify by checking your agent's command directory (see the table in Exercise 1 step 5). For example, Claude Code users would run:

```
ls .claude/commands/opsx/
```

You should now see additional commands alongside the core ones: `new`, `continue`, `ff`, `verify`, and possibly others (the file extension varies by agent). These are the expanded profile commands you will use throughout this lab.

4. **Understand what changed.** The profile switch regenerated your agent's command files. Your core commands (**propose**, **apply**, **archive**, **explore**) still work exactly as before. The expanded profile adds options. It does not change the existing workflow.

In this lab, you will use:

- **new**: create an empty change folder (no artifacts generated)
- **continue**: generate the next artifact in the dependency chain
- **apply**: same as before, implement from the task checklist
- **verify**: validate implementation against specs
- **archive**: same as before, merge specs and archive

## 4.5. Exercise 3: Scaffold and Build the Proposal

Now you will begin building the feature using the expanded workflow. Instead of generating all artifacts at once, you start with an empty scaffold and build one artifact at a time.

1. **Scaffold the change.** Open your agent's chat panel and invoke the **new** command with `add-ingredient-cost-model` as the change name (see the

**Command syntax** block at the top of this lab for the exact form your agent uses).

This creates `openspec/changes/add-ingredient-cost-model/` with an empty structure. No proposal, no specs, no design, no tasks. You control what gets generated and when.

Verify the folder was created:

```
ls openspec/changes/add-ingredient-cost-model/
```

2. **Generate the proposal.** Now generate the first artifact. The proposal. Provide the feature context so the AI has a clear starting point:

Read the specs in openspec/specs/ for context on the current system.

This change adds an ingredient cost model and financial dashboard.

Ingredient cost model:

- Define base ingredients with unit costs (espresso per shot, milk per oz, chocolate per oz, whipped cream per serving, alternative milk per oz)
- Map each drink/size to ingredient quantities – e.g., a Large Neural Network Latte uses 2 espresso shots + 12oz milk. Small and medium drinks typically use 1 shot; large drinks use 2.
- Calculate COGS per order from the recipe ingredients
- Customization surcharges (like extra shot) must also add their ingredient cost to COGS, not just the customer price

Financial dashboard:

- Display alongside the order board showing total revenue, total COGS, and gross margin (revenue minus COGS)
- Per-order breakdown showing revenue, COGS, and margin per order
- Updates live as orders are placed and completed

Out of scope: supplier management, ingredient inventory tracking, purchase ordering, historical reporting, multi-location support.

Then run the continue command. Generate the proposal only and stop --

I will review it before continuing.

After pasting the context block, invoke the **continue** command (see the **Command syntax** block at the top of this lab). The agent should generate the **proposal only**. Read it before moving on.

#### Note

AI agents sometimes generate more than the proposal in this step (creating specs, design, or tasks alongside it). If that happens, see "**continue` generated more than one artifact**" in Common Issues at

the end of this lab for guidance on how to recover the review-gate workflow.

3. **Review gate. The proposal.** Open `openspec/changes/add-ingredient-cost-model/proposal.md` and ask these questions:

- **Is the scope right?** Are ingredients and recipes treated as one change, or should this be split into two? (One change is correct here. The financial dashboard needs both ingredients and recipes to calculate COGS. Splitting them would create a partial feature.)
- **Are customization costs in scope for COGS?** The proposal should explicitly mention that customizations like an extra shot add ingredient cost, not just surcharge revenue. If it does not, add this to the proposal.
- **Is anything in scope that should not be?** Watch for scope creep: the AI might add inventory tracking, supplier management, or cost-over-time analytics that you did not ask for. Remove these if present. Edit the proposal directly in VS Code if needed. The next artifact (specs) will be generated from this reviewed proposal, so corrections here prevent downstream errors.

#### Note

This is the first review gate. In previous labs, you reviewed all four artifacts after generation. Here, you review one artifact at a time. If the proposal scope is wrong, you fix it *before* the specs are generated. Rather than discovering the problem in the specs and having to fix both.

## 4.6. Exercise 4: Build Specs and Design with Review Gates

Continue generating artifacts one at a time. Each invocation of the **continue** command produces the next artifact in the dependency chain.

1. **Generate the specs.** Invoke the **continue** command in your agent's chat panel.  
The agent reads the reviewed proposal and generates the delta specs.



2. **Review gate. The specs.** Open the generated spec files in `openspec/changes/add-ingredient-cost-model/specs/`. Ask these questions:

- **Are COGS calculation requirements concrete enough to implement?** Look for specific language about how COGS is calculated: sum of (ingredient quantity x unit cost) for each ingredient in the recipe. If the spec just says "calculate COGS" without defining the formula, the AI will guess.
- **Do the specs say how customization costs factor into COGS?** An extra espresso shot adds revenue (\$0.75 to the customer) \_and\_ cost (\$0.40 for the espresso). The spec should require that COGS includes customization ingredient costs, not just base recipe costs. **This is a common gap. Check for it specifically.**
- **Does the spec define what "gross margin" means?** Revenue minus COGS? As a dollar amount, a percentage, or both? If it is ambiguous, the AI picks one. Make it explicit.
- **Are ingredient quantities specified?** The spec should define recipe mappings with concrete numbers (shots, ounces) per drink and size, or describe where they come from (a configuration file, code constants). Edit the specs to fill any gaps. In particular, if the customization COGS requirement is missing or vague, add it now. For example:

```
Requirement: COGS includes customization ingredient costs
COGS for an order MUST include the ingredient cost of any customizations. For example, an extra espresso shot adds the cost of one espresso shot to the order's COGS, in addition to the $0.75 surcharge added to the customer price.
```

#### Tip

Fixing this gap now is the expanded workflow paying off. If you had used **propose**, this gap would have propagated into the design and tasks. You would discover it during verification (or worse, in the running application) and have to trace it back through multiple artifacts.

3. **Generate the design.** Invoke the **continue** command again. The agent reads the reviewed proposal and reviewed specs to generate the design.

4. **Review gate. The design.** Open `openspec/changes/add-ingredient-cost-model/design.md` and ask:

- **Does the data model handle size-dependent recipes?** Large drinks use 2 espresso shots while small and medium use 1. The design should show how recipe mappings vary by size.
  - **Where do recipe mappings live?** In code as constants? In a configuration file like `menu.json`? In a separate data file? Any approach works, but the design should make a clear decision. (Code constants or a separate JSON file are both reasonable for this project.)
  - **Does the design address how the dashboard updates live?** The dashboard shows metrics that change as orders are placed and completed. The design should describe whether this is polling, server-sent events, or recalculated on each page render. (For BeanBotics with its simple in-memory store, polling or recalculating on render is the simplest approach.)
- Edit the design if needed. As with previous labs, watch for implementation details that belong in specs or tasks. Variable names, type signatures, and file paths are signs that the design is at the wrong abstraction level.

5. **Generate the tasks.** Invoke the **continue** command once more.  
The agent reads all reviewed artifacts and generates the task checklist.

6. **Review gate. The tasks.** Open `openspec/changes/add-ingredient-cost-model/tasks.md` and ask:

- **Are tasks ordered by dependency?** Ingredient definitions must come before recipe mappings. Recipe mappings must come before COGS calculations. COGS calculations must come before the financial dashboard. If the tasks are out of order, reorder them.
  - **Are financial dashboard tasks after the ingredient/recipe tasks they depend on?** The dashboard cannot show COGS until the COGS calculation exists.
  - **Is each task small enough to verify independently?** A task like "implement the full financial dashboard" is too broad. Better: "add COGS calculation endpoint," "add dashboard HTML structure," "wire up live updates."
- Edit the task list if needed. The task order directly controls how **apply** works through the implementation.

## 4.7. Exercise 5: Implement and Verify

With all four artifacts reviewed and refined, you are ready to implement. This is the same **apply** step from previous labs. The expanded workflow changes how you *plan*, not how you *implement*.

1. **Clear context and apply.** Clear the planning conversation before implementing. The artifacts are finalized in files. The agent reads them fresh. Use your agent's "clear context" or "new session" affordance (for example, Claude Code users would run `/clear`). Then start implementation by invoking the **apply** command.

Watch the agent work through the task checklist. Because you reviewed and refined each artifact before this step, the agent is working from a plan that has been corrected at each stage.

2. **Verify in the browser.** Refresh the application at `http://localhost:8000` (hard refresh: `Ctrl + Shift + R`).

### What to check:

- **Financial dashboard visible:** You should see a financial panel alongside (or near) the order board showing revenue, COGS, and gross margin. The exact layout depends on the design.
- **Dashboard starts at zero:** With no completed orders, revenue, COGS, and margin should all be \$0.00 (or the dashboard may show data only after orders are placed. Either is acceptable).
- **Place a test order:** Order a Large Neural Network Latte (\$6.50) with no customizations. Does the dashboard update? Check whether COGS appears for the order.  
If the dashboard does not appear, hard refresh again. If it still does not appear, check whether all tasks completed. Ask the agent: "Are all tasks in tasks.md checked off?"

3. **Run verify.** Now invoke the **verify** command to check the implementation against the specs.

The agent reads the specs and checks the code for three things:

- **Completeness:** Are all requirements implemented? Are all tasks done?
- **Correctness:** Does the implementation match the spec's intent?
- **Coherence:** Do design decisions show up in the code?

Review the verify output. It may surface issues you did not catch during manual testing. Particularly requirements that were implemented incorrectly or not at all.

4. **The customization COGS test.** This is the critical verification. Place a specific test order and check whether the COGS calculation is correct:
- Place an order for a **Large Neural Network Latte** (\$6.50) with an **extra espresso shot** (+\$0.75)
  - Advance the order through the lifecycle to **completed**
  - Check the financial dashboard

**Calculate the expected COGS by hand:**

A Large Neural Network Latte recipe uses 2 espresso shots + 12oz milk (based on the ingredient model). The extra shot customization adds 1 more espresso shot:

```
Base recipe COGS:
 2 espresso shots x $0.40/shot = $0.80
 12oz milk x $0.05/oz = $0.60
 Base COGS = $1.40

Customization COGS:
 1 extra espresso shot x $0.40 = $0.40

Total COGS: = $1.80

Revenue (pre-tax subtotal):
 Large Neural Network Latte $6.50
 Extra shot surcharge + $0.75
 Revenue = $7.25

Gross margin: $7.25 - $1.80 = $5.45
```

#### Note

Your exact ingredient costs and recipe quantities may differ from the example above. Check your spec files in `openspec/changes/add-ingredient-cost-model/specs/` for ingredient unit costs, and look at the recipe data file (likely `backend/data/ingredients.json` or `backend/data/recipes.json`, depending on your design) for the per-drink, per-size ingredient quantities. Use those values for

your hand calculation. The important thing is that the COGS includes the extra shot's ingredient cost, not just the base recipe cost.

### What to check on the dashboard:

- Does the COGS for this order include the cost of the extra espresso shot? Or does it only show the base recipe COGS?
- If COGS is \$1.40 (base only) instead of \$1.80 (base + customization), the spec gap was not caught. The implementation ignores customization ingredient costs

5. **Fix any issues.** If verify or manual testing found problems:

**Spec gap (customization COGS missing):** If the COGS calculation ignores customization ingredients, the spec may not have required it explicitly. Open the spec file in `openspec/changes/add-ingredient-cost-model/specs/` and add or strengthen the requirement:

```
Requirement: COGS includes customization ingredient costs
COGS for an order MUST include the ingredient cost of every
customization. An extra espresso shot MUST add the cost of one
espresso shot ($0.40) to the order's COGS. The COGS displayed on
the financial dashboard MUST reflect the full ingredient cost
including customizations.
```

Then clear context (using your agent's clear-context affordance) and invoke the **apply** command again.

**Code bug (spec is correct but implementation is wrong):** If the spec clearly requires customization COGS but the code ignores it, point the agent at the discrepancy:

```
The spec requires that COGS includes customization ingredient
costs.
The extra shot should add $0.40 to the COGS. Check the COGS
calculation and fix it to include customization ingredients.
```

**Dashboard not updating:** If the numbers are correct but the dashboard does not refresh, this is likely a frontend issue. Ask the agent to check whether the dashboard polls for updates or recalculates on render.

After fixing, verify again to confirm the fix worked.

## 4.8. Exercise 6: Archive and Review the Spec Suite

1. **Archive.** Once the implementation is verified, invoke the **archive** command.
2. **Examine the updated spec suite.**

```
ls openspec/specs/
```

The spec suite has grown again. You should see new spec files covering the ingredient cost model and financial dashboard domains alongside the existing specs from previous labs.

Count the domain areas now covered:

- Drink size selection
  - Order status lifecycle
  - Drink customizations
  - Order confirmation and receipts
  - Ingredient cost model and financial dashboard (this lab)
- This is becoming a comprehensive behavioral description of the BeanBotics system. Each domain spec reflects the current truth about that part of the system.

3. **Trace the evolution.**

```
ls openspec/changes/archive/
```

Five archived changes with timestamps. Each one records what changed, why, and how. A new developer joining the project could read the spec suite to understand *what* the system does and browse the archive to understand *how* it evolved.

Open one of the earlier archived changes (e.g., the size selection change) and compare its scope to the ingredient cost model change you just completed. Notice how the complexity grew: the first change had a handful of requirements and straightforward scenarios. This change has interrelated requirements across ingredients, recipes, COGS, and the dashboard. The expanded workflow scaled with the complexity.

## 4.9. Reflection Questions

Take a few minutes to consider these questions.

1. **Which artifact did you change the most during review?** Was it the proposal, specs, design, or tasks? What kind of issue did you catch? Would the **propose** command (all-at-once) have produced the same artifact differently, or would the issue have been there too?
2. **Did catching issues between artifacts save time?** Compare the expanded workflow to what would have happened if all four artifacts were generated at once. Would you have caught the same issues during a post-generation review? Would it have been harder to fix them?
3. **What happened with the customization COGS test?** Did your spec require it? Did the implementation get it right? If the spec was silent on customization COGS, at what point in the workflow could you have caught it earliest. Proposal, specs, design, or tasks?
4. **When would you use the continue command vs. the propose command?** Based on your experience in this lab and previous labs, where is the line? What signals tell you a feature needs incremental artifact generation rather than all-at-once?
5. **How has the spec suite changed?** Compare the spec suite now (five domain areas) to where it was after the first feature (one domain area). Is this a document you would hand to a new team member? What would they learn from reading it?
6. **Would you have built this feature differently?** If you did this lab again, what would you change about your review at each gate? Were there questions you wish you had asked earlier?

## 4.10. Common Issues

### "Menu stuck on 'Loading menu...' and never populates"

The browser tab is left over from a previous lab. It was open before the current server started, so the menu fetch is pending against a server that no longer exists. Do a hard refresh with `Ctrl + Shift + R`. If that does not fix it, confirm the URL is `http://localhost:8000/` and that `uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000` is still running in the terminal.

### "New commands not appearing after profile switch"

After running `openspec config profile`, the new commands should appear in your agent immediately. If they do not, verify the files exist in your agent's command directory (see the table in Exercise 1 step 5). For example, Claude Code users would run:

```
ls .claude/commands/opsx/
```

You should see `new`, `continue`, `ff`, and `verify` alongside the core commands (the file extension varies by agent). If they are missing, run `openspec update` to sync the current config into the project.

This especially applies if the TUI showed the expanded workflows already checked and reported **"No config changes"**: that means your global OpenSpec config already has them selected (from a prior install or run), so the TUI skipped regenerating the project files. `openspec update` is the fix.

If the command files exist in your agent's command directory but the new commands still do not appear in your agent's UI, the agent may have cached the command list at startup. Reload your editor window (for VS Code-based agents: `Ctrl + Shift + P` → **Developer: Reload Window**; for standalone IDEs like Cursor, Windsurf, or Kiro, restart the application) and the new commands will appear.

### "`continue` generated more than one artifact"

The agent should generate one artifact per **continue** invocation, but sometimes generates several at once. Typically because the agent decided to run the skill repeatedly. Anything beyond `proposal.md` was created from an **un-reviewed** proposal, which defeats the review-gate workflow.

To recover:

1. Review and edit `proposal.md` first as instructed in Exercise 3.
2. If your edits substantively changed the proposal, delete the prematurely-generated artifacts ( `specs/`, `design.md`, `tasks.md` ) inside `openspec/changes/add-ingredient-cost-model/` and invoke the **continue** command again for each one in Exercise 4. The regenerated artifacts will be based on your corrected proposal.
3. If the proposal needed only minor edits or was already correct, you can keep the existing artifacts and review them at their gates in Exercise 4.



### "`continue` generates the wrong artifact"

Continue generates the next artifact based on what is already complete. If it generates specs when you expected the proposal, check whether a proposal already exists in the change folder. Run `openspec status --change add-ingredient-cost-model` to see which artifacts are done and which are next.

### "Financial dashboard does not appear"

Do a hard refresh with `Ctrl + Shift + R`. If the dashboard still does not appear, check whether all tasks in `tasks.md` are checked off. The dashboard may be one of the later tasks. Ask the agent: "Are all tasks complete? If not, which ones are remaining?" Then invoke the **apply** command again.

### "COGS shows \$0.00 for all orders"

The COGS calculation likely depends on recipe mappings. Check whether the recipe data was created correctly. Ask the agent to check whether the recipe mappings exist and whether the COGS calculation reads them.

### "Dashboard does not update after placing orders"

The dashboard may need a page refresh to show updated numbers, depending on the implementation. If the spec requires live updates, check whether the frontend polls the backend or recalculates on demand. A simple fix: ask the agent to add polling that refreshes the dashboard data every few seconds.

### "Verify reports issues that seem minor"

The **verify** command checks comprehensively and may flag issues that are not critical. Use your judgment: if verify says a requirement is not implemented but the behavior is correct (just implemented differently than the spec describes), that may be acceptable. If verify says a requirement is missing entirely, that is worth fixing.

### "Archive fails with validation errors"

Run `openspec validate` to see the specific issues. The most common cause is unchecked tasks in `tasks.md`. Ensure all tasks are marked complete before archiving.

## 4.11. Challenge Exercise (Optional)

If you finish early, try one of these exercises.

### 4.11.1. Calculate Margins for Multiple Orders

Place three different orders with different drinks, sizes, and customizations:

1. Small Deep Learning Doppio (\$3.50), no customizations
2. Medium Machine Learning Mocha (\$6.00), with whipped cream (+\$0.50)
3. Large Neural Network Latte (\$6.50), with extra shot (\$0.75) and oat milk (\$0.60)

Advance all three to completed. Calculate the expected COGS for each by hand using the ingredient costs and recipe mappings in your spec. Compare your hand calculations to the dashboard. Do the numbers match?

This exercise tests whether the COGS calculation handles variation correctly. Different drinks with different recipes, different sizes with different ingredient quantities, and different customizations.

### 4.11.2. Write a Spec Scenario by Hand

Write a new Given/When/Then scenario for the financial dashboard that is not already in the spec:

```
Scenario: Dashboard reflects cancelled order correctly
- **Given** two orders have been placed with total revenue of $13.75
 and total COGS of $3.50
- **When** one order with revenue $7.25 and COGS $1.80 is cancelled
- **Then** the dashboard SHALL show:
 - Total Revenue: $6.50
 - Total COGS: $1.70
 - Gross Margin: $4.80
```

Add this scenario to the relevant spec file in `openspec/specs/`. This is practice for writing scenarios by hand. Does the current implementation handle cancelled orders correctly in the dashboard? If not, you have found a gap.

 **Note**

Whatever you build in the challenge exercises is for learning purposes. It will not carry forward to the next lab, which starts from a known checkpoint state.

 **Tip**

Your work in `~/LabFiles/expanded-workflow-spec-lifecycle/beanbotics/` is preserved after the lab. If you want to restart the lab from the clean starting state, re-extract the course materials ZIP (or copy the pristine folder from a fresh extract).

## 4.12. Context Hygiene Recap

This lab introduced a new context pattern: **multiple context sessions within a single feature**.

In previous labs, you used one context session for each propose-apply cycle (or cleared between two cycles). In this lab, the expanded workflow naturally involves multiple context interactions:

- **Planning phase:** the **new** command, then multiple **continue** invocations with review and editing between each
- **Implementation phase:** Clear context, then invoke **apply** with finalized artifacts
- **Verification phase:** the **verify** command followed by any fixes

The key insight: the **continue** command reads existing artifacts from files, not from conversation history. This means you *can* clear context between continue steps if the conversation gets cluttered. Each continue step rehydrates from the files. In practice, most developers keep context during the planning phase (the conversation about review decisions is useful) and clear before the apply phase (clean context for implementation).

There is no single right answer. The question to ask is: "Is the context window helping or hurting right now?" If the conversation has accumulated debugging tangents, failed approaches, or off-topic discussions, clear it. If the conversation has useful review decisions that inform the next artifact, keep it.

# Real-World Patterns & Spec Management

## **LAB 5**

- 
- ✓ Fix a small issue without using the spec workflow, exercising the judgment call
  - ✓ Review the accumulated spec suite and identify which specs contain pricing assumptions
  - ✓ Write a proposal and delta specs by hand using ADDED, MODIFIED, and REMOVED sections
  - ✓ Have the AI agent review hand-written specs and identify gaps
  - ✓ Implement dynamic pricing using the hand-written specs as the starting point
  - ✓ Trace the full evolution of the BeanBotics spec suite across the course

## 5.1. Prerequisites

Before starting this lab, ensure you have:

- Completed the Expanded Workflow & Spec Lifecycle lab (or have the checkpoint files)
- **Python 3.10+** with `pip`
- **Node.js 18+** with `npm`
- **VS Code** (or your preferred editor) with one of the supported AI coding agents installed, signed in, and configured (see the **Command syntax** note below)
- **OpenSpec CLI** installed ( `openspec --version` to verify)

### Note

Command syntax depends on your agent  
Throughout this course, when prose says "invoke the **propose** command," substitute the command name into the pattern for your agent:

| Agent                                                | Example: <b>propose</b>    |
|------------------------------------------------------|----------------------------|
| Claude Code                                          | <code>/opsx:propose</code> |
| Cursor, Windsurf, GitHub Copilot, Google Antigravity | <code>/opsx-propose</code> |
| Amazon Q, AWS Kiro                                   | <code>@opsx-propose</code> |

Apply the same pattern to every other command ( `apply` , `archive` , `continue` , ...).

## 5.2. Introduction

In every lab so far, you used the spec workflow: propose (or new/continue), apply, archive. Every change introduced new behavior with ADDED sections in the delta specs. The AI generated the artifacts and you reviewed them.

This lab is different in three ways:

1. **You will fix something without SDD.** A small cosmetic issue that does not warrant the spec workflow. This is the "when SDD is overkill" judgment call from earlier in the course, practiced for real.
2. **You will write specs by hand.** Instead of the AI generating proposals and delta specs, you will write them yourself. The AI becomes your reviewer, not your author.
3. **You will modify existing specs.** The dynamic pricing feature changes behavior that has been specified since earlier labs. For the first time, your delta specs will include MODIFIED and REMOVED sections alongside ADDED.

**The feature:** Dynamic pricing with peak and off-peak tiers. During peak hours (7-9am, 11:30am-1:30pm), drinks are full price. During off-peak hours, base drink prices are 20% off. A time simulation slider in the UI lets you control the current time to test both tiers instantly.

### 5.3. Exercise 1: Set Up Your Working Copy

In this exercise, you will open this lab's BeanBotics folder in VS Code, create a virtual environment, install dependencies, and verify the app runs. The starting state is the completed state from the previous lab. BeanBotics with size selection, order lifecycle, customizations, confirmation/receipts, and the ingredient cost model with financial dashboard.

1. **Open this lab's BeanBotics folder in VS Code.** The starter state for this lab is at `~/LabFiles/real-world-patterns-spec-management/beanbotics/`. Open it as its own VS Code workspace:

```
code ~/LabFiles/real-world-patterns-spec-management/beanbotics
```

If VS Code is already open on a different folder, open a new window (File → New Window) and open this folder in the new window, or close the current window and reopen with the command above.

#### ⚠ Important

For the rest of this lab, **all work happens in this VS Code window** with `beanbotics` as the project root. Your agent and the OpenSpec commands only work when VS Code (or your agent's IDE) is opened to

the directory where OpenSpec is initialized. The checkpoint includes expanded profile commands from a previous lab. **new**, **continue**, **ff**, and **verify** are all available alongside the core commands (see **Command syntax** above for how these are invoked in your agent).

2. **Set up the Python environment.** Open a terminal inside the VS Code window ( `Ctrl` + ``` ) and run:

```
uv venv
uv pip install -r requirements.txt
```

3. **Clean up any previously running server.** If you still have a server running from an earlier session, stop it now. Find the terminal where it is running and press `Ctrl` + `C` , or restart your terminal session if you cannot locate it.

4. **Run the app.**

```
uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
```

Verify the app is running at <http://localhost:8000> . You should see the full BeanBotics UI: menu with size selection and customizations, order confirmation, status-grouped order queue, receipt viewing, and the financial dashboard with revenue/COGS/margin.

5. **Review your starting state.**

```
ls openspec/specs/
```

You should see **eleven spec files** covering five domain areas built across the earlier labs: drink size selection, order status lifecycle, drink customizations, order confirmation/receipts, and the ingredient cost model with financial dashboard.

```
ls openspec/changes/archive/
```

Five archived changes with timestamps. The complete history of BeanBotics so far.

## 5.4. Exercise 2: Fix Without SDD

This exercise is deliberately short. You will find a small cosmetic issue in the app and fix it by talking to the agent directly. No **propose** command, no specs, no artifacts.

1. **Find a cosmetic issue.** Open the app at <http://localhost:8000> and look at it with a critical eye. After several labs of AI-generated implementation, there are likely rough edges. Look for things like:
  - Inconsistent spacing or alignment between sections
  - A label or heading that could be clearer
  - Color or font inconsistencies between the menu, order queue, and financial dashboard
  - A button that does not quite match the style of other buttons
  - Missing or unclear column headers in the financial dashboard tablePick **one** small issue. It should be something you can describe in a single sentence.

2. **Fix it directly.** Open your agent's chat panel and describe the issue. No OpenSpec commands. Just tell the agent what to fix:

The [describe your issue] – fix it to [describe what you want].

For example: "The financial dashboard table header text is hard to read against the dark background. Make it match the style of the order queue headers."

The agent fixes it. Done.

3. **Verify and reflect.** Refresh the browser ( **Ctrl** + **Shift** + **R** ) and confirm the fix looks right.

Now ask yourself: **would a spec have helped here?**

- The intent was self-evident. You could describe the fix in one sentence
- No behavioral requirement changed. The system works the same way
- No spec in [openspec/specs/](#) needs updating
- The fix is purely visual

This is the "when SDD is overkill" heuristic from earlier in the course, practiced for real. The answer is clear: just fix it and move on. Save the spec workflow for behavioral changes.





 **Note**

This fix creates **spec drift** in one sense: the code changed but the specs did not. The specs only cover *behavior*, not visual styling, and a cosmetic change that does not affect behavior is outside the spec's scope. No drift has occurred.

## 5.5. Exercise 3: Identify Pricing Assumptions in the Spec Suite

Before writing delta specs, you need to know what you are modifying. This exercise reviews the accumulated specs to identify where pricing assumptions are baked in.

1. **Read the specs that touch pricing.** Open each of these spec files and note where pricing is mentioned:

```
cat openspec/specs/drink-size-selection/spec.md
cat openspec/specs/order-receipt/spec.md
cat openspec/specs/financial-dashboard/spec.md
```

As you read, note requirements that assume **fixed prices**. For example:

- The size selection spec says the menu displays prices per size, but assumes the price is always the base price from `menu.json`
  - The receipt spec shows itemized pricing with a fixed base price
  - The financial dashboard shows revenue computed from order totals, but does not distinguish between pricing tiers
2. **Identify what changes.** For the dynamic pricing feature (full price during peak, 20% off during off-peak), note which existing requirements need to change:
    - **Menu display:** During off-peak, prices need to show the original price (strikethrough) and the discounted price. During peak, prices show normally.
    - **Order placement:** The backend needs to calculate the price based on the active pricing tier, not just the static menu price.
    - **Receipts:** Off-peak orders need a discount line item showing the savings.

- **Financial dashboard:** Revenue needs to distinguish peak vs. off-peak to show the impact of discounting.  
Also note what is entirely **new** (not modifying existing specs):
- Pricing tier definitions (peak windows, discount percentage)
- Time simulation slider for testing
- Backend pricing tier determination  
And what is being **removed**:
- The assumption that drink prices are always the fixed base size price from `menu.json`  
You will use these notes in the next exercise when writing the delta specs.

## 5.6. Exercise 4: Write Specs by Hand

This is the core exercise of the lab. You will write a proposal and delta specs **without AI generation**. The AI becomes your reviewer after you finish writing.

1. **Scaffold the change.** In your agent's chat panel, invoke the **new** command with the argument `add-dynamic-pricing` (see **Command syntax** above for the exact form).  
This creates `openspec/changes/add-dynamic-pricing/` with an empty structure.
2. **Write the proposal.** Open `openspec/changes/add-dynamic-pricing/proposal1.md` in VS Code and write it by hand. Use the structure you have seen across the earlier labs of AI-generated proposals:
  - **Why:** BeanBotics uses fixed pricing regardless of time of day. Coffee shops typically offer off-peak discounts to attract customers during slow periods. Dynamic pricing lets the business run full prices during peak hours and discount during off-peak.
  - **What changes:** Pricing tier definitions (peak and off-peak), time simulation for testing, modified menu display, modified order pricing, modified receipts, modified financial dashboard.
  - **Scope boundaries:** What is out of scope? (Suggestions: per-drink discount overrides, customer-facing tier schedule, historical pricing data, different discount percentages for different times.)  
Keep it skimmable in two minutes.

3. **Write the delta specs.** This is the most important step. You will create delta spec files that include ADDED, MODIFIED, and REMOVED sections. The first time you are modifying existing specs rather than only adding new ones.

### Create the Scaffolding

First, create the folder and file structure for your delta specs. Each domain that this change touches gets its own folder with a `spec.md` file inside it. This matches the pattern you have seen in every previous lab's archived changes.

Create these folders and files inside `openspec/changes/add-dynamic-pricing/specs/`:

```
mkdir -p openspec/changes/add-dynamic-pricing/specs/pricing-
tiers
mkdir -p openspec/changes/add-dynamic-pricing/specs/menu-display
mkdir -p openspec/changes/add-dynamic-pricing/specs/order-
pricing
mkdir -p openspec/changes/add-dynamic-pricing/specs/order-
receipt
mkdir -p openspec/changes/add-dynamic-pricing/specs/financial-
dashboard

touch openspec/changes/add-dynamic-pricing/specs/pricing-tiers/
spec.md
touch openspec/changes/add-dynamic-pricing/specs/menu-display/
spec.md
touch openspec/changes/add-dynamic-pricing/specs/order-pricing/
spec.md
touch openspec/changes/add-dynamic-pricing/specs/order-receipt/
spec.md
touch openspec/changes/add-dynamic-pricing/specs/financial-
dashboard/spec.md
```

Your change folder should now look like this:

```

openspec/changes/add-dynamic-pricing/
├─ proposal.md ← written in Step 2
├─ specs/
│ └─ pricing-tiers/
│ └─ spec.md ← ADDED: tier
definitions, time slider
├─ menu-display/
│ └─ spec.md ← MODIFIED:
strikethrough + discounted price
├─ order-pricing/
│ └─ spec.md ← MODIFIED: tier-based
price calculation
├─ order-receipt/
│ └─ spec.md ← MODIFIED: discount
line item
├─ financial-dashboard/
│ └─ spec.md ← MODIFIED: peak vs.
off-peak breakdown

```

### Note

In this lab, we are providing the scaffolding so you can focus on the hard part: writing spec content with ADDED, MODIFIED, and REMOVED sections. In a real project, you would decide this structure yourself. Choosing which domains to create based on which parts of the system the change touches. Look at `openspec/specs/` to see the existing domain structure, and create delta spec folders for each domain your change affects. If a change introduces an entirely new domain (like pricing tiers), create a new folder. If it modifies an existing domain (like receipts), create a folder that maps to the existing spec.

### Fill in the Spec Files

Writing five delta specs from scratch would take longer than the lab allows. Instead, we are providing three of the five delta specs for you and asking you to write the remaining two. This lets you focus your time on the skill this exercise teaches. Writing ADDED, MODIFIED, and REMOVED sections. Without spending the full lab on spec authoring alone.

In a real project, you would write all of them. The three we provide are realistic examples of what the AI would generate if you invoked **continue**: study them before writing your own.

**Provided:** `menu-display/spec.md`

Copy the following into

`openspec/changes/add-dynamic-pricing/specs/menu-display/spec.md` :

## # Delta Spec: Menu Display – Dynamic Pricing

### ## MODIFIED

#### ### Requirement: Menu cards display size options with prices

**\*\*Previous:\*\*** Each menu card SHALL display a size selector showing all available sizes (small, medium, large) with their corresponding prices.

**\*\*Updated:\*\*** Each menu card SHALL display a size selector showing all available sizes (small, medium, large) with pricing that reflects the active pricing tier. During peak hours, prices display normally. During off-peak hours, each size option SHALL show the original price with a strikethrough and the discounted price beside it. A pricing tier indicator SHALL be visible on the menu showing whether peak or off-peak pricing is active.

#### #### Scenario: Menu during peak hours

- **\*\*WHEN\*\*** the active time is within a peak window (7:00am-9:00am or 11:30am-1:30pm)
- **\*\*THEN\*\*** each size option shows its base price normally with no discount indicators
- **\*\*AND\*\*** the menu displays a "Peak Pricing" indicator

#### #### Scenario: Menu during off-peak hours

- **\*\*WHEN\*\*** the active time is outside all peak windows
- 
- **\*\*THEN\*\*** each size option shows the original base price with a strikethrough and the discounted price (base price minus 20%) beside it
- **\*\*AND\*\*** the menu displays an "Off-Peak Pricing – 20% Off" indicator

#### #### Scenario: Prices update when time changes

- **\*\*WHEN\*\*** the customer adjusts the time simulation slider from a peak time to an off-peak time
- **\*\*THEN\*\*** the menu prices update immediately to show strikethrough and discounted prices without a page reload

**Provided:** `order-pricing/spec.md`

Copy the following into `openspec/changes/add-dynamic-pricing/specs/  
order-pricing/spec.md`:

## # Delta Spec: Order Pricing – Dynamic Pricing

### ## MODIFIED

#### ### Requirement: Order is placed with the selected size

**\*\*Previous:\*\*** When the customer clicks the order button, the system SHALL send the selected size to the API instead of always sending "medium".

**\*\*Updated:\*\*** When the customer clicks the order button, the system SHALL send the selected size and the current simulated time to the API. The backend SHALL determine the active pricing tier from the simulated time and calculate the base drink price accordingly: full price during peak hours, 20% off during off-peak hours. Customization surcharges (extra shot, milk alternatives, whipped cream) SHALL NOT be discounted – only the base drink price is affected.

#### #### Scenario: Order placed during peak hours

- **\*\*WHEN\*\*** the customer places an order for a Large Neural Network Latte (\$6.50) at 8:00am (peak)
- **\*\*THEN\*\*** the backend uses \$6.50 as the base price
- **\*\*AND\*\*** the order total reflects the full base price plus any customization surcharges

#### #### Scenario: Order placed during off-peak hours

- **\*\*WHEN\*\*** the customer places an order for a Large Neural Network Latte (\$6.50) at 3:00pm (off-peak)
- 
- **\*\*THEN\*\*** the backend applies a 20% discount to the base price:  
 $\$6.50 \times 0.80 = \$5.20$
- **\*\*AND\*\*** the order total reflects \$5.20 plus any customization surcharges at full price

#### #### Scenario: Customization surcharges are not discounted

- 
- **\*\*WHEN\*\*** the customer places an order at 3:00pm (off-peak) for a Large Neural Network Latte with an extra espresso shot (+\$0.75)
- **\*\*THEN\*\*** the base price is discounted to \$5.20 but the extra shot remains \$0.75
- **\*\*AND\*\*** the pre-tax subtotal is \$5.95

#### #### Scenario: Price is locked at order placement time



```
-
 WHEN the customer places an order at 8:59am (peak) and the
 time crosses into off-peak (9:01am) before the order is
 completed
 - **THEN** the order retains the peak price from when it was
 placed

REMOVED

Assumption: Drink prices are always the base size price
Reason: Drink prices now vary by time of day. The base size
price in menu.json represents the full (peak) price. During off-
peak hours, a 20% discount is applied to the base drink price.
The fixed-price assumption is replaced by tier-based pricing.
```

**Provided:** `financial-dashboard/spec.md`

Copy the following into `openspec/changes/add-dynamic-pricing/specs/  
financial-dashboard/spec.md`:

## # Delta Spec: Financial Dashboard – Dynamic Pricing

### ## MODIFIED

#### ### Requirement: Aggregate financial metrics display

**\*\*Previous:\*\*** The frontend SHALL display a financial dashboard showing three aggregate metrics: total revenue (sum of all order totals), total COGS (sum of all order COGS), and gross margin (revenue minus COGS). The dashboard SHALL appear alongside the order board.

**\*\*Updated:\*\*** The financial dashboard SHALL display aggregate metrics broken down by pricing tier. The dashboard SHALL show: total revenue, total COGS, and gross margin as before, plus a breakdown showing peak revenue vs. off-peak revenue. This allows the business to see the impact of off-peak discounting on overall revenue and margins.

#### #### Scenario: Dashboard distinguishes peak and off-peak revenue

-

**\*\*WHEN\*\*** orders have been placed during both peak and off-peak hours

-

**\*\*THEN\*\*** the dashboard displays total revenue with a breakdown showing peak revenue and off-peak revenue separately

#### #### Scenario: Off-peak margins are lower than peak margins

- **\*\*WHEN\*\*** the same drink is ordered during peak and off-peak hours

- **\*\*THEN\*\*** the off-peak order shows lower revenue but the same COGS

-

**\*\*AND\*\*** the off-peak gross margin is lower than the peak gross margin

#### ### Requirement: Per-order financial breakdown

**\*\*Previous:\*\*** The dashboard SHALL include a per-order breakdown table showing each order's revenue (order total), COGS, and margin (revenue minus COGS).

**\*\*Updated:\*\*** The per-order breakdown table SHALL include a pricing tier column indicating whether each order was placed

during peak or off-peak hours. For off-peak orders, the revenue column reflects the discounted price.

#### Scenario: Per-order row shows pricing tier

- **\*\*WHEN\*\*** an order was placed during off-peak hours
- **\*\*THEN\*\*** the per-order row displays "Off-Peak" in the pricing tier column
- **\*\*AND\*\*** the revenue reflects the discounted base price plus any full-price customization surcharges

**Your Turn:** `pricing-tiers/spec.md`

Open `openspec/changes/add-dynamic-pricing/specs/pricing-tiers/spec.md` and write it yourself. This is an **ADDED** spec. Entirely new requirements, not modifying anything that exists.

Cover these requirements:

- **Pricing tier definitions:** Define the two tiers. Peak windows (7:00am-9:00am and 11:30am-1:30pm) at full price, off-peak (all other hours) at 20% off base drink prices. Specify that customization surcharges are NOT discounted.
- **Time simulation slider:** The UI must include a time-of-day slider for selecting the simulated current time. The system uses the slider value to determine the active tier. Menu and pricing must update immediately when the slider changes.
- **Backend time parameter:** The API accepts an optional time parameter that overrides server time for pricing tier determination. Write Given/When/Then scenarios for at least two of these requirements. Use the ADDED section format and RFC 2119 keywords (MUST, SHALL, MUST NOT) as you have seen in every previous lab.

**Your Turn:** `order-receipt/spec.md`

Open `openspec/changes/add-dynamic-pricing/specs/order-receipt/spec.md` and write it yourself. This is a **MODIFIED** spec. You are changing existing receipt behavior.

Before writing, read the current receipt spec to see what you are modifying:

```
cat openspec/specs/order-receipt/spec.md
```

You need to modify the receipt to handle off-peak discounts. Cover:

- **MODIFIED. Receipt line items:** Off-peak orders must include an "Off-Peak Discount (20%)" line item showing the discount amount between the base price and the customizations. The subtotal, tax, and total must reflect the discounted price. Peak orders display as they do today (no changes).

Include the **Previous** and **Updated** text for the requirement you are modifying. Quote the previous version from the existing spec in

```
openspec/specs/order-receipt/spec.md
```

. Write at least one Given/

When/Then scenario showing a receipt with the discount line item.

#### Tip

Do not worry about getting the specs perfect on your first pass. Write what you can, and the AI will review all five specs in the next step. The three we provided and the two you wrote. The point is to practice writing, not to produce a flawless document.

4. **Have the AI review your specs.** This reverses the dynamic from every previous lab. You wrote the specs; now the AI reviews them.

```
Read the proposal and delta specs I wrote in
openspec/changes/add-dynamic-pricing/. Compare them
against the existing specs in openspec/specs/.
```

Review for:

- Gaps: requirements I missed or scenarios I didn't cover
- Ambiguities: language that could be interpreted multiple ways
- Conflicts: new requirements that contradict existing ones
- Edge cases: boundary conditions I didn't address

```
Tell me what's missing or unclear.
```

Read the AI's feedback carefully. It may identify:

- **Missing scenarios:** What happens when a customer is in the confirmation step and the time crosses a tier boundary? (The price should be locked at the time the order is placed, not when it is confirmed.)
  - **Rounding ambiguity:** How should the 20% discount be rounded? Round the discount amount, or round the final price?
  - **Dashboard specifics:** How exactly should peak vs. off-peak revenue be displayed? Separate totals? Percentage breakdown?
  - **Time simulation defaults:** What pricing tier is active when the page first loads? Does the control default to peak or off-peak?
- Edit your specs based on the feedback. You do not need to address every suggestion. Use your judgment about which gaps matter.

## 5.7. Exercise 5: Implement Dynamic Pricing

With the proposal and specs written and reviewed, generate the remaining artifacts and implement.

1. **Generate design and tasks.** You have hand-written the proposal and specs. In your agent's chat panel, invoke the **ff** command (see **Command syntax** above) to generate the design and tasks from them.

The agent reads your hand-written proposal and specs and generates `design.md` and `tasks.md`.

Review the generated design and tasks briefly. Because your specs are the input, the design and tasks should align with what you wrote. If something looks off, edit it.

2. **Clear context and apply.** Clear your agent's conversation context (for example, with a `/clear` command or by starting a new session), then invoke the **apply** command in your agent's chat panel (see **Command syntax** above).

Watch the agent implement from the task list. The specs it follows are the ones **you wrote**: this is the first time the implementation is driven entirely by your hand-authored specifications.

3. **Verify in the browser.** Refresh the application at `http://localhost:8000` (hard refresh: `Ctrl + Shift + R`).

**What to check:**

- **Time simulation control visible:** You should see a control in the UI for switching between peak and off-peak pricing. The exact UI depends on the implementation. It may be a slider, a toggle, buttons, or a time picker.
- **Peak pricing:** Switch to peak pricing. The menu shows full prices with no discount indicators.
- **Off-peak pricing:** Switch to off-peak pricing. The menu shows original prices with strikethrough and discounted prices (20% off base drink price).

Test a specific order in off-peak mode:

- a. Switch to **off-peak** pricing
- b. Order a **Large Neural Network Latte** (\$6.50 full price, \$5.20 with 20% off)
- c. Add an **extra shot** (+\$0.75. NOT discounted)
- d. Confirm and place the order

**Expected receipt values:**

|                                  |         |
|----------------------------------|---------|
| Large Neural Network Latte ..... | \$6.50  |
| Off-Peak Discount (20%) .....    | -\$1.30 |
| Extra espresso shot .....        | \$0.75  |
| Subtotal                         | \$5.95  |
| Tax (8.5%)                       | \$0.51  |
| Total                            | \$6.46  |

**Note**

The discount applies only to the base drink price (\$6.50 x 0.20 = \$1.30), not to customization surcharges. Tax is calculated on the discounted subtotal. Your exact receipt format may vary depending on how you specified the layout in your delta specs.

Now test peak mode:

- e. Switch to **peak** pricing
- f. Place the same order (Large Neural Network Latte + extra shot)
- g. The receipt should show no discount line item. Full price (\$6.50 + \$0.75 = \$7.25 subtotal)

4. **Run verify.** Invoke the **verify** command in your agent's chat panel (see **Command syntax** above).  
Because you wrote the specs by hand, verify checks the implementation against **your** requirements. This is a good test of your spec writing. If verify finds gaps, they may be gaps in your specs, not just the implementation.
5. **Check the financial dashboard.** Place a few orders across both pricing tiers (switch between peak and off-peak) and check the financial dashboard:
  - Does the dashboard distinguish peak vs. off-peak revenue?
  - Do the margins differ? (Off-peak revenue is lower but COGS is the same, so margin shrinks)
  - Are the totals correct when you add peak and off-peak orders together?
6. **Fix any issues.** If the implementation does not match your specs:
  - **Spec gap:** Your hand-written spec missed something. Add the requirement and re-apply.
  - **Code bug:** The spec is clear but the implementation is wrong. Point the agent at the discrepancy.
  - **Time simulation not working:** Check whether the time control sends the selected time to the backend. Ask the agent to verify the time simulation endpoint.
7. **Archive.** Once verified, invoke the **archive** command in your agent's chat panel (see **Command syntax** above).  
This time, archive does more than append. It **modifies** existing requirements in `openspec/specs/` and **removes** the fixed-price assumption. Check the results:

```
cat openspec/specs/drink-size-selection/spec.md
```

The spec should now reflect the updated pricing behavior. Requirements you MODIFIED should show the new text. Requirements you REMOVED should be gone.

## 5.8. Exercise 6: Trace the Full Evolution

1. **The final spec suite.**

```
ls openspec/specs/
```

The spec suite now covers **six domain areas** built across the course: drink size selection, order status lifecycle, drink customizations, order confirmation with receipts, the ingredient cost model with financial dashboard, and dynamic pricing. The number of spec files may exceed six. AI agents often split a domain into multiple files (for example, separating UI specs from data specs, or COGS calculation from recipe mapping). A new developer joining this project could read this directory and understand every behavioral contract in the system.

## 2. The complete archive.

```
ls openspec/changes/archive/
```

Six archived changes. Each one records the proposal, specs (with ADDED, MODIFIED, or REMOVED sections), design, and tasks. The archive is a complete audit trail of the system's evolution.

3. **Read a MODIFIED spec.** Open the archived dynamic pricing change and look at one of your MODIFIED delta specs. Then open the corresponding file in `openspec/specs/` and confirm the modification was applied correctly. The previous text should be gone, replaced by your updated requirement. This is the spec lifecycle in action: propose, implement, archive, and the source of truth evolves.

## 5.9. Reflection Questions

Take a few minutes to consider these questions.

1. **How did it feel to fix something without SDD?** Was the judgment call obvious? Would you have hesitated in a real project, or is the line clear between "just fix it" and "use the spec workflow"?
2. **What was hardest about writing specs by hand?** Was it the MODIFIED sections (quoting the previous version), the scenarios (concrete values), or the scope boundaries (what to include vs. leave out)? How did writing compare to reviewing AI-generated specs?
3. **What did the AI's review of your specs catch?** Were the gaps it found things you would have caught during implementation anyway, or were they genuinely non-obvious?



4. **How did MODIFIED and REMOVED feel in practice?** Was it clear which existing requirements needed to change? Did you have trouble deciding what to put in REMOVED vs. MODIFIED?
5. **Look at the final spec suite.** Could a new developer read it and understand the system? What would they learn that the code alone does not communicate? What would still be unclear?
6. **What would you do differently next time?** If you could redo this lab, would you write the specs differently? Would you ask the AI for more or less help?

## 5.10. Common Issues

### "Menu stuck on 'Loading menu...' and never populates"

The browser tab is left over from a previous lab. It was open before the current server started, so the menu fetch is pending against a server that no longer exists. Do a hard refresh with `Ctrl + Shift + R`. If that does not fix it, confirm the URL is `http://localhost:8000/` and that `uv run uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000` is still running in the terminal.

### "Time simulation control does not appear"

Do a hard refresh with `Ctrl + Shift + R`. If the control still does not appear, check whether all tasks in `tasks.md` are checked off. The time simulation may be one of the later tasks. Re-invoke **apply** if tasks are incomplete.

### "Prices do not change when switching between peak and off-peak"

Check whether the time control is sending the selected time to the backend. Open the browser developer tools ( `F12` ) and watch the network tab as you switch pricing tiers. You should see requests being made. If no requests are sent, the frontend is not wired up to the control. Ask the agent to check the event handler.

### "Off-peak discount is applied to customization surcharges"

Your spec should state that customization surcharges are not discounted. If it does and the implementation discounts them anyway, this is a code bug. Point the agent at the spec. If your spec did not specify this, it is a spec gap. Add the requirement and re-apply.

**"Receipt does not show discount line item"**

Check your MODIFIED spec for the receipt. Did you specify that off-peak orders must include an "Off-Peak Discount" line item? If the spec is there but the receipt does not show it, the implementation missed a requirement. Invoke **verify** to confirm, then ask the agent to fix it.

**"Financial dashboard does not distinguish peak vs. off-peak"**

This may not have been implemented if your spec was not specific enough about how the distinction should appear. Check your MODIFIED spec for the financial dashboard. If it just says "distinguish peak and off-peak" without specifying how (separate columns, separate rows, percentage breakdown), the AI may have interpreted it minimally.

**"Archive changes existing specs incorrectly"**

Run `openspec validate` before archiving. If validation passes but the merged specs look wrong after archive, check your MODIFIED sections. The previous text must match what is currently in `openspec/specs/` for the merge to work correctly.

**"The ff command regenerates the proposal and specs I wrote"**

If **ff** overwrites your hand-written files, the change folder may not have recognized them as complete. Check that your proposal and spec files are saved and non-empty. If needed, invoke **continue** instead to generate only the design, then invoke **continue** again for tasks.

## 5.11. Challenge Exercise (Optional)

If you finish early, try one of these exercises.

### 5.11.1. Add a Third Pricing Tier

Write a delta spec by hand for a "late night" pricing tier (after 8pm) with a 30% discount. This exercises writing MODIFIED sections for requirements you just modified. The pricing tier definitions need to be updated from two tiers to three.

Consider: does this change require MODIFIED sections for the menu display, receipt, and dashboard specs you just wrote? Or are those specs already general enough to handle a third tier without modification?

### 5.11.2. Calculate the Margin Impact

Place three orders across pricing tiers:

1. Large Neural Network Latte during **peak** (full price \$6.50)
2. Large Neural Network Latte during **off-peak** (\$5.20)
3. Large Neural Network Latte during **peak** again (full price \$6.50)

All three have the same COGS (same recipe, same ingredients). Calculate the gross margin for each by hand. How much does the off-peak discount reduce the margin?

This is the financial reality of discounting. Revenue drops but costs stay the same. The financial dashboard should make this visible.

#### Note

Whatever you build in the challenge exercises is for learning purposes. It will not carry forward. This is the final lab.

#### Tip

Your work in `~/LabFiles/real-world-patterns-spec-management/beanbotics/` is preserved after the lab. If you want to restart the lab from the clean starting state, re-extract the course materials ZIP (or copy the pristine folder from a fresh extract).

## 5.12. Context Hygiene Recap

This lab used several context patterns from across the course:

- **No context needed** (Exercise 2). The cosmetic fix was small enough that a single conversational message handled it

- **Clean context for spec review** (Exercise 4). The AI reviewed your hand-written specs in a focused session
- **Clean context for implementation** (Exercise 5). Cleared before apply so the agent reads finalized artifacts fresh

The final pattern is worth reflecting on: **specs are persistent context that survives any context window**. Conversations disappear when you close the session. Specs persist in files. The spec suite you built across the course is the system's memory. It captures intent, decisions, constraints, and behavioral contracts that no chat history could preserve.

When you return to a project after a break, you do not need to reconstruct what you were thinking. You read the specs. When a new team member joins, they do not need to interview everyone who worked on the project. They read the specs. When an AI agent starts a fresh session with no conversation history, it reads the specs and knows what the system does.

This is the payoff of Spec-Driven Development: **durable, structured context that outlives any single session**.